[UNIVERSITY NAME]

[Faculty / School of Computing and Information Technology]

[Department of Computer Science / Software Engineering]

[UNIVERSITY LOGO PLACEHOLDER]

SOFTWARE ANALYSIS AND DESIGN (SAD) DOCUMENTATION

Smart Child Daily Activity Tracking System

An Offline Family Dashboard for Monitoring Children's Daily Habits

Prepared by: [Student Name]

Registration Number: [Registration Number]

Supervisor: [Supervisor Name]

Programme: [BSc Software Engineering / Computer Science]

Academic Year: 2025/2026

August 2026

DECLARATION

I hereby declare that this Software Analysis and Design (SAD) documentation titled “Smart Child Daily Activity Tracking System” is my own original work and has not been submitted for any degree or examination in any other university. All sources of information used in this document have been duly acknowledged.

Name: _____ Signature: _____

Date: _____

Supervisor: _____ Signature:

_____

Date: _____

APPROVAL PAGE

This Software Analysis and Design (SAD) documentation has been examined and is approved for submission in partial fulfilment of the requirements of the [Programme] at [University Name].

Supervisor	Name & Signature: Date:
Head of Department	Name & Signature: Date:
Dean / Coordinator	Name & Signature: Date:

ABSTRACT

The Smart Child Daily Activity Tracking System is an offline family dashboard designed for households in which children do not own smartphones. The system runs on a single shared tablet or touchscreen placed in the home; children simply select their own profile, while the parent administers the entire system. This document presents the complete software analysis and design of the system. The analysis began with a review of existing child habit-tracking applications, which confirmed a gap: no existing product combines shared-device operation, complete offline functionality, a parent-verification workflow, culturally aware scheduling and periodic family reporting. Requirements were elicited through a questionnaire and interviews and specified as twenty-eight functional requirements, eleven non-functional requirements and ten business rules. The design includes two actors, twelve parent use cases and nine child use cases, a context diagram, DFDs at levels zero, one and two, an entity-relationship model, a thirteen-table MySQL schema, a domain class diagram, sequence, activity and state diagrams, and five workflow diagrams covering coins, approvals, rewards and the parent and child processes. The architecture is a five-layer stack: a Flutter client, a Laravel REST API, business services, Eloquent ORM and MySQL, all communicating over the offline home network. Eight screen wireframes define the user interface. A complete project management plan schedules the work over four weeks, budgets the project at USD 280.50, and identifies risks, quality measures, communication and testing strategies. The document concludes with an implementation plan and is ready for direct development and university submission.

Keywords: Offline family dashboard, child activity tracking, gamification, coin economy, parent verification, Flutter, Laravel, MySQL.

ACKNOWLEDGEMENTS

I wish to express my sincere gratitude to my supervisor, [Supervisor Name], for the guidance, encouragement and constructive feedback provided throughout the preparation of this document. My appreciation also goes to my lecturers, colleagues and the parents who participated in the questionnaire and interviews that shaped the requirements of the system. Finally, I am grateful to my family for their patience, support and understanding during this project.

TABLE OF CONTENTS

CHAPTER ONE 1

INTRODUCTION 1

1.1 Background of the Study 1

1.2 Problem Statement 2

1.3 Aim of the Study 2

1.4 Objectives of the Study 2

1.5 Research Questions 3

1.6 Significance of the Study 3

1.7 Scope of the Study 4

1.8 Limitations of the Study 4

1.9 Definition of Terms 4

1.10 Organisation of the Document 5

CHAPTER TWO 6

LITERATURE REVIEW 6

2.1 Introduction 6

2.2 Theoretical Background 6

2.3 Review of Existing Systems 6

2.3.1 Habitica 6

2.3.2 ChoreMonster 7

2.3.3 OurHome 7

2.3.4 Parental Control and Monitoring Applications 7

2.4 Comparison of Existing Systems 7

2.5 Research Gap 8

2.6 Proposed Solution 8

2.7 Advantages of the Proposed System 8

2.8 Disadvantages of the Proposed System 8
2.9 Overall Comparison: Existing vs Proposed 9
2.10 Summary 9
CHAPTER THREE 10
SYSTEM ANALYSIS AND DESIGN 10
3.1 Introduction 10
3.2 Current System Analysis 10
3.2.1 Overview of the Current (Manual) System 10
3.2.2 Problems with the Current System 10
3.3 Proposed System Requirements 11
3.4 Functional Requirements 11
3.5 Non-Functional Requirements 13
3.6 Business Rules 14
3.7 Actors 14
3.8 Use Case Modelling 15
3.8.1 Use Case Diagram 15
3.8.2 Use Case Descriptions 15
3.8.3 Activity Diagram 20
3.8.4 Sequence Diagram 20
3.8.5 State Diagram 21
3.8.6 Workflow Diagrams 22
3.9 Data Flow Analysis 27
3.9.1 Context Diagram 27
3.9.2 Data Flow Diagram – Level 0 28
3.9.3 Data Flow Diagram – Level 1 29
3.9.4 Data Flow Diagram – Level 2 30

3.10 Data Design 31
3.10.1 Entity-Relationship Diagram 31
3.10.2 Database Design and Data Dictionary 31
3.10.3 Database Relationship Diagram 34
3.11 Class Diagram 35
3.12 Navigation Structure 36
3.13 System Architecture 37
3.14 Deployment Architecture 38
3.15 User Interface Design (Wireframes) 38
3.15.1 Design Guidelines 38
3.15.2 Login / PIN Screen 39
3.15.3 Parent Dashboard 40
3.15.4 Child Dashboard 41
3.15.5 Task Screen 42
3.15.6 Reports Screen 43
3.15.7 Coins Screen 44
3.15.8 Rewards Screen 45
3.15.9 Settings Screen 46
3.16 Summary 46
CHAPTER FOUR 47
PROJECT MANAGEMENT 47
4.1 Introduction 47
4.2 Development Methodology 47
4.3 Work Breakdown Structure (WBS) 47
4.4 Project Schedule 48
4.4.1 Gantt Chart 48

4.4.2 Timeline 49
4.4.3 Milestones 49
4.5 Resource Planning 50
4.5.1 Hardware Requirements 50
4.5.2 Software Requirements 50
4.5.3 Human Resources 50
4.6 Budget Estimation 51
4.7 Risk Analysis 51
4.7.1 Risk Identification 51
4.7.2 Risk Matrix 52
4.8 Quality Plan 53
4.9 Communication Plan 53
4.10 Testing Plan 53
4.10.1 Testing Strategy 53
4.10.2 Sample Test Cases 54
4.11 Requirement Traceability Matrix 54
4.12 Summary 55
CHAPTER FIVE 56
IMPLEMENTATION PLAN AND CONCLUSION 56
5.1 Introduction 56
5.2 Implementation Strategy 56
5.3 Testing Strategy 56
5.4 Deployment Plan 57
5.5 Maintenance Plan 57
5.6 Future Improvements 57
5.7 Expected Results 57

5.8 Conclusion 58

REFERENCES 59

APPENDIX A: QUESTIONNAIRE 60

APPENDIX B: INTERVIEW QUESTIONS 62

APPENDIX C: GLOSSARY 63

APPENDIX D: ABBREVIATIONS AND ACRONYMS 64

LIST OF TABLES

1 : *Definition of Key Terms* 4

2 : *Comparison of Existing Child Habit Tracking Systems* 7

3 : *Summary Comparison – Existing Systems versus Proposed System* 9

4 : *Problems of the Current Manual System* 10

5 : *Functional Requirements* 11

6 : *Non-Functional Requirements* 13

7 : *Business Rules* 14

8 : *Actors of the System* 14

9 : *Data Dictionary of the MySQL Schema* 31

10 : *Work Breakdown Structure* 47

11 : *Project Timeline* 49

12 : *Project Milestones* 49

13 : *Hardware Requirements* 50

14 : *Software Requirements* 50

15 : *Budget Estimation and Cost Breakdown* 51

16 : *Risk Identification and Analysis* 51

17 : *Risk Matrix (Likelihood × Impact)* 52

18 : *Communication Plan* 53

19 : *Testing Strategy* 53

20 : *Sample Test Cases* 54

21 : *Requirement Traceability Matrix* 54

22 : *Parent Requirements Questionnaire* 60

23 : *Glossary of Terms* 63

24 : *Abbreviations and Acronyms* 64

LIST OF FIGURES

1 : *Use Case Diagram of the System* 15

2 : *Activity Diagram – Task Completion and Verification* 20

3 : *Sequence Diagram – Task Completion and Parent Approval* 21

4 : *State Diagram – Assignment Lifecycle* 22

5 : *Coin Workflow* 23

6 : *Approval Workflow* 24

7 : *Reward Workflow* 25

8 : *Parent Workflow* 26

9 : *Child Workflow* 27

10 : *Context Diagram of the System* 28

11 : *DFD Level 0* 28

12 : *DFD Level 1 – Main Processes* 29

13 : *DFD Level 2 – Verification and Approval* 30

14 : *Entity-Relationship Diagram (Logical Design)* 31

15 : *Database Relationship Diagram* 34

16 : *Class Diagram (Domain Model)* 35

17 : *Navigation Structure of the Application* 36

18 : *System Architecture – Five-Layer Stack* 37

19 : *Deployment Architecture* 38

20 : *Login / PIN Screen – Wireframe* 39

21 : *Parent Dashboard – Wireframe* 40

22 : *Child Dashboard – Wireframe* 41

23 : *Task Screen – Wireframe* 42

24 : *Reports Screen – Wireframe* 43

25 : *Coins Screen – Wireframe* 44

26 *: Rewards Screen – Wireframe* 45

27 *: Settings Screen – Wireframe* 46

28 *: Gantt Chart – Project Schedule (03–31 August 2026)* 49

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Children in the modern family are surrounded by digital technology, yet the tools designed to support their daily routines rarely consider families in which the child does not own a smartphone. Popular habit-tracking and gamified parenting applications assume that every child can install an application, create an account and carry a personal mobile device. In reality, many households, particularly those concerned about screen time, online safety and data cost, prefer that their children use technology only inside the home and under direct parental supervision.

The Smart Child Daily Activity Tracking System addresses this gap by providing an offline family dashboard that runs on a single shared tablet or touchscreen device placed in the home. Children are never expected to log in to a personal phone. Instead, a child simply chooses his or her own profile from the tablet and the system adapts the entire interface to that child. Parents, acting as administrators of the family system, manage every aspect of the dashboard: they create children, define tasks, set schedules, assign coin values, approve or reject completed work, create rewards and badges, and monitor performance through daily, weekly, monthly and yearly reports.

The system combines behaviour-modification theory with practical family management. Children earn coins for completing tasks, collect badges for consistent effort, and exchange coins for rewards that the parent defines. Two verification models are supported: self-check, in which a child receives coins immediately, and parent approval, in which a child receives temporary coins that are only retained if the parent later approves the task. Because the entire platform operates on a local network without an internet connection, the system is private, secure and usable in homes with limited or no broadband connectivity.

Technologically, the system is a three-tier application. The front-end is built with Flutter and runs on the family tablet. The back-end is a Laravel (PHP) REST API served on a home computer acting as a local server, and all persistent data is stored in a MySQL database. All communication takes place over the home Wi-Fi network, and no data ever leaves the household.

1.2 Problem Statement

Existing child habit and chore applications are designed around the assumption that the child owns a smartphone. This assumption creates a number of problems for families who do not wish to give their children personal devices:

- **Device ownership barrier:** children without smartphones cannot use most existing applications.
- **Account management overload:** managing separate accounts and passwords for each child is burdensome and unsafe.
- **Internet dependency:** cloud-based applications stop working when the home network is offline.
- **Weak supervision:** parents cannot verify that a task was actually completed because most applications rely on the child's own claim without an approval channel.
- **Lack of flexible scheduling:** fixed chore lists do not support weekend, Friday-only or custom schedules such as attending Jummah prayer or visiting grandparents.
- **No family-level reporting:** parents cannot compare children or generate period-based progress reports easily.
- **Reward handling is arbitrary:** there is no transparent coin ledger, so earning and spending of rewards is unclear to both parent and child.

Consequently, parents who want to encourage responsibility and good habits must fall back on paper charts, verbal reminders and manual record keeping, which are difficult to maintain consistently and produce no useful statistics. There is therefore a clear need for an offline, shared-device system that places control firmly in the hands of the parent while motivating the child through a transparent coin, badge and reward economy.

1.3 Aim of the Study

The aim of this study is to analyse, design and document a Smart Child Daily Activity Tracking System — an offline family dashboard that runs on one shared home tablet, allows parents to manage tasks, coins, badges and rewards, and allows children to track and complete their daily activities without owning a personal smartphone.

1.4 Objectives of the Study

The specific objectives of the study are to:

1. analyse the requirements of an offline, shared-device family activity tracking system and document them as clear functional and non-functional requirements;
2. design the complete architecture of the system, including use cases, data flow diagrams, entity-relationship model, class diagram, navigation structure, deployment model and database schema;
3. design a motivating coin, badge and reward economy with self-check and parent-approval verification workflows;
4. design periodic reporting (daily, weekly, monthly and yearly) that includes completed, rejected, pending and missed tasks, coins, badges, completion rate and top-child ranking;
5. prepare a professional project management plan covering schedule, resources, budget, risks, quality, communication and testing; and
6. produce a complete Software Analysis and Design (SAD) document ready for university submission.

1.5 Research Questions

The study is guided by the following research questions:

1. RQ1: What are the functional and non-functional requirements of an offline shared-tablet system for tracking children's daily activities in homes where children do not own smartphones?
2. RQ2: How can a coin, badge and reward economy be designed to motivate children while remaining fully controlled and verifiable by parents?
3. RQ3: How can self-check and parent-approval verification workflows be modelled so that coins are awarded fairly and consistently?
4. RQ4: What data must be stored and analysed to produce meaningful daily, weekly, monthly and yearly progress reports?
5. RQ5: How can the system be architected to operate entirely within a home local network without an internet connection?

1.6 Significance of the Study

The study is significant in several ways. For parents, it offers a practical, private and supervised tool that improves children's habits without introducing personal smartphones into the household. For children, it provides a fair and transparent reward system that makes

responsibility visible and enjoyable. For the field of software engineering, the study demonstrates the complete analysis and design of a small-scale, offline-first family application using industry-standard techniques, including structured analysis, object-oriented modelling and relational database design. Finally, the document serves as a complete academic submission that shows the application of software engineering principles from requirements capture through to implementation planning.

1.7 Scope of the Study

The scope of this study covers the complete analysis and design of the Smart Child Daily Activity Tracking System. The design covers two user roles — the parent (administrator) and the child. It includes the management of children, tasks, schedules, coins, rewards, badges, verification, messages, notifications, settings and periodic reports. The system is designed to run entirely offline on a home local network using a Flutter client, a Laravel PHP REST API and a MySQL database. The document does not include actual implementation code; it is restricted to the software analysis and design deliverables: requirements, use cases, DFDs, ER model, class and sequence diagrams, navigation, architecture, deployment, database design, UI wireframes, and the project management plan.

1.8 Limitations of the Study

The following limitations apply to the study:

- **Single-household deployment:** the system is designed for one home and one local server, not for multi-household cloud hosting.
- **No internet or cloud features:** remote monitoring outside the home network is deliberately out of scope.
- **One shared device:** the design assumes a single shared tablet; multi-device family use is not covered.
- **Design phase only:** the document presents analysis and design; the actual coding, deployment and operational testing are covered in the implementation plan but not performed here.
- **Generalisability:** reward values, badge criteria and report thresholds are configurable defaults that each family can adjust.

1.9 Definition of Terms

1*: Definition of Key Terms*

Term	Definition
Administrator	The parent role that manages all configuration, verification and reporting in the system.
Assignment	A single instance of a task assigned to a specific child for a specific date.
Badge	A visual achievement awarded by the parent for consistent performance, such as “Reading Hero”.
Child	A non-administrator user who selects a profile on the shared tablet and completes tasks.
Coin	The internal currency earned for completing tasks and used to redeem rewards.
Completion Rate	The percentage of assigned tasks that were completed and verified within a reporting period.
Kiosk Mode	The child-facing interface that requires only profile selection, not a personal login.
Offline Local Network	A home Wi-Fi network where the tablet, server and database communicate without internet access.
Parent Approval	A verification type in which coins are temporary until the parent approves the task.
Redemption	The process of exchanging coins for a reward, which the parent must approve.
Self-Check	A verification type in which the child’s completion is accepted immediately and coins are awarded at once.
Task	An activity (daily, weekly, monthly, custom, weekend or specific-days) that a child must complete.
Verification	The process of confirming that a completed task is accepted or rejected.

1.10 Organisation of the Document

This document is organised into five chapters. Chapter One introduces the study. Chapter Two presents the literature review, including existing systems, a comparison and the research gap. Chapter Three contains the full system analysis and design, including requirements, use cases, DFDs, database design, UML diagrams, architecture and UI wireframes. Chapter Four presents the project management plan, covering methodology, schedule, resources, budget, risk, quality, communication and testing. Chapter Five describes the implementation plan and concludes the document. The references and appendices follow.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews the body of work and products related to child habit tracking and gamification in the family context. It first discusses the theoretical background, then examines representative existing systems, compares them against the needs of families whose children do not own smartphones, identifies the research gap and finally presents the proposed solution and its advantages and disadvantages.

2.2 Theoretical Background

The design of the proposed system draws on established behavioural and design theory. Gamification, defined as the use of game design elements in non-game contexts, has been shown to increase motivation for routine tasks when rewards are immediate, predictable and meaningful. Token economies, used widely in child behaviour management, operate on the principle that positive behaviours are strengthened when they are followed by a tangible token that can later be exchanged for a valued reward. The proposed system operationalises this through coins (tokens), rewards (back-up reinforcers) and badges (achievement markers).

Habit formation theory also informs the task-scheduling design. Research on habit formation emphasises consistency and cue-repetition; therefore the system supports daily, weekly, monthly, weekend, Friday-only and custom schedules so that routines are matched to the natural rhythm of the family and the religious or cultural obligations of the household.

2.3 Review of Existing Systems

A number of commercial and research applications target the same broad goal of motivating children. The most representative are reviewed below.

2.3.1 Habitica

Habitica is a gamified habit tracker that turns daily routines into an online role-playing game. Users create a character and lose or gain health and experience based on completing or missing habits. It is widely praised for its strong gamification, but it is designed for individual adult users, requires an account and an internet connection, and offers no parent approval or verification workflow suitable for young children.

2.3.2 ChoreMonster

ChoreMonster allows parents to assign chores, children complete them on their own devices, and the children earn points that can be exchanged for rewards chosen by the parent. It captures the coin-and-reward model well, but it is a cloud-based mobile application that assumes each child has (or shares) a device with internet access. It does not support an offline local network, and verification is limited to the child's self-report with a simple parent confirm action.

2.3.3 OurHome

OurHome focuses on family organisation, assigning tasks to each family member and awarding points automatically when tasks are marked complete. It includes a family feed and rewards. Although it supports multiple children on shared devices, it still requires a cloud account and internet connection, and the scheduling options (daily, specific days, custom) are less structured than the requirement for Friday-only and weekend-only routines described in this project.

2.3.4 Parental Control and Monitoring Applications

Applications such as FamilyTime and Qustodio focus on screen-time limits, location tracking and usage monitoring rather than positive habit formation. They provide parents with powerful oversight but do not address daily routines, task verification or a rewards economy, and they are fundamentally cloud services.

2.4 Comparison of Existing Systems

2: Comparison of Existing Child Habit Tracking Systems

Feature	Habitica	ChoreMonster	OurHome	Parental Control Apps
Primary purpose	Habit / RPG tracker	Chores & rewards	Family tasks & points	Monitoring & limits
Shared tablet (no child phone)	No	No	Partial	No
Offline / local network	No	No	No	No
Parent approval & rejection	No	Partial	No	No
Coin / reward economy	Yes (XP/HP)	Yes	Yes	No
Badges	Yes	Partial	No	No
Flexible scheduling (Friday-only, weekend)	Partial	No	Partial	No
Periodic family reports	Personal only	No	Limited	Usage reports
Top-child comparison	No	No	No	No
Runs entirely in the home	No	No	No	No

2.5 Research Gap

The review shows that no existing application combines all of the following characteristics: (1) operation on a single shared tablet without any child smartphone, (2) complete offline operation on a home local network, (3) a full parent-approval workflow in which temporary coins are automatically removed when a task is rejected, (4) flexible cultural and religious scheduling such as Friday-only and weekend-only tasks, (5) a transparent coin ledger with bonus and deduction records, and (6) daily, weekly, monthly and yearly family reports with completion rate and top-child ranking. This combination represents a genuine research and product gap that the proposed system fills.

2.6 Proposed Solution

The proposed Smart Child Daily Activity Tracking System is an offline family dashboard running on one shared tablet. A Laravel PHP REST API and a MySQL database run on a home computer, and all communication stays on the local network. Children select their profile and interact with child-friendly screens, while parents manage the entire system through an administrator interface. Tasks carry schedules and coin values; verification is either self-check or parent approval; coins, badges and rewards are fully parent-controlled; and the system generates periodic reports with charts and analytics.

2.7 Advantages of the Proposed System

- Removes the need for children to own smartphones.
- Works completely offline, preserving privacy and avoiding data-cost concerns.
- Gives parents full control of tasks, verification, coins, rewards and badges.
- Provides transparent handling of temporary coins with automatic removal on rejection.
- Supports cultural schedules (Friday, weekend, custom and specific days).
- Generates daily, weekly, monthly and yearly reports with completion rate and top-child ranking.
- Uses a familiar, child-friendly kiosk interface with avatars and badges.
- Is built on a maintainable three-tier architecture (Flutter / Laravel / MySQL).

2.8 Disadvantages of the Proposed System

- Single-household scope — no remote monitoring when away from home.

- Requires a home computer to act as the local server, which must remain powered.
- Manual data backup responsibility rests with the parent.
- Multi-device family use is not supported in the initial design.
- Limited to the analysis and design documented here until implementation and operational testing are completed.

2.9 Overall Comparison: Existing vs Proposed

3: Summary Comparison – Existing Systems versus Proposed System

Criterion	Existing Systems	Proposed System
Child device requirement	Child smartphone / account	Shared home tablet only
Connectivity	Internet required	Offline local network
Verification	Self-report or none	Self-check + parent approval with rejection reason
Coin integrity	Manual / unclear	Ledger with automatic deduction on rejection
Scheduling	Simple	Daily, weekly, monthly, weekend, Friday, custom, specific days
Reporting	None or usage only	Daily–yearly reports, completion rate, top child
Data privacy	Cloud storage	Stays in the home

2.10 Summary

This chapter reviewed the theoretical foundations and representative existing systems. The comparison confirms a clear gap for an offline, shared-tablet family dashboard with parent verification, flexible cultural scheduling and periodic reporting. The proposed system is designed specifically to close this gap, and the following chapter presents its full analysis and design.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents the full analysis and design of the Smart Child Daily Activity Tracking System. It begins with an analysis of the current manual system and its problems, then defines the functional and non-functional requirements and business rules. The system is then modelled using structured and object-oriented techniques: actors and use cases, activity, sequence and state diagrams, workflows, context diagram, DFDs, ER model, database design, class diagram, navigation structure, system architecture and deployment model. The chapter closes with UI wireframes for every major screen.

3.2 Current System Analysis

3.2.1 Overview of the Current (Manual) System

In the target households, children's daily activities are currently managed manually. Parents create paper charts or simple lists that show expected tasks for each day. A typical chart might list morning routines such as brushing teeth, praying, making the bed, completing homework and playing outside, with boxes to tick for each day of the week. When a child completes a task, the parent or the child places a tick. At the end of the week the parent inspects the chart and may reward the child informally with treats, extra play time or money. Reminders are delivered verbally, and there is no record of the child's consistency over time.

3.2.2 Problems with the Current System

4: Problems of the Current Manual System

#	Problem	Effect
P1	Paper charts are easily lost, torn or ignored.	No reliable long-term record.
P2	Tracking is manual and time-consuming for parents.	Parents abandon tracking within weeks.
P3	There is no consistent reward rule.	Children lose motivation and trust.
P4	Verification is impossible on paper.	Children may tick tasks that were not done.
P5	No statistics or completion rates exist.	Parents cannot measure improvement.
P6	Schedules such as Friday-only or weekend-only are not handled.	Cultural routines are missed.

P7	Comparison across children is difficult.	Fairness between siblings is unclear.
P8	No reminders are possible.	Routines depend entirely on memory.

3.3 Proposed System Requirements

The proposed system was specified through requirement elicitation that included a review of existing applications, a structured questionnaire completed by parents, and interviews with parents and a child-development specialist (see Appendices A and B). The resulting requirements are grouped into functional requirements (FR), non-functional requirements (NFR) and business rules (BR).

3.4 Functional Requirements

The functional requirements describe the behaviour the system must provide. Each requirement has a unique identifier used in the Requirement Traceability Matrix in Chapter Four.

5: Functional Requirements

ID	Requirement	Category
FR-01	The parent shall log in securely using a username/password and a four-digit PIN.	Authentication
FR-02	Only the parent shall be able to access administrator functions.	Authentication
FR-03	The parent shall add, edit and delete child profiles (name, avatar, date of birth).	Children
FR-04	The child shall select his or her own profile from the shared tablet without a login.	Children
FR-05	The parent shall create, edit, delete and deactivate tasks.	Tasks
FR-06	Each task shall have a title, description, schedule type, repeat days and coin value.	Tasks
FR-07	The system shall support daily, weekly, monthly, custom, weekend and specific-day schedules.	Scheduling
FR-08	The system shall generate assignments (task instances) for each child on the relevant dates.	Scheduling

FR-09	Each task shall be assigned either self-check or parent-approval verification.	Verification
FR-10	Self-check completion shall award coins immediately and mark the task verified.	Verification
FR-11	Parent-approval completion shall award temporary coins and mark the task “pending verification”.	Verification
FR-12	The parent shall approve or reject pending tasks; rejection shall require a reason.	Verification
FR-13	Rejection shall automatically remove the temporary coins and record a ledger deduction.	Verification
FR-14	Uncompleted tasks past their due date shall be marked “missed”.	Verification
FR-15	The parent shall add bonus coins and remove coins for any child with a recorded reason.	Coins
FR-16	Every coin credit and debit shall be recorded in a transaction ledger.	Coins
FR-17	The parent shall create, edit and deactivate rewards with a coin cost.	Rewards
FR-18	The child shall view available rewards and request redemption if coins are sufficient.	Rewards
FR-19	Redemption requests shall require parent approval before coins are deducted.	Rewards
FR-20	The parent shall create badges and assign them to children.	Badges
FR-21	The child shall view earned badges on the dashboard.	Badges
FR-22	The system shall generate daily, weekly, monthly and yearly reports.	Reports
FR-23	Reports shall include completed, rejected, pending and missed tasks, coins, badges, completion rate and top child.	Reports
FR-24	The parent shall view charts and analytics for any child and period.	Reports
FR-25	The parent shall send messages that the child can read on the tablet.	Messages

FR-26	The system shall record reminders and show notifications on the tablet.	Notifications
FR-27	The parent shall configure settings including PIN, notification preferences and task defaults.	Settings
FR-28	All child-facing screens shall use a child-friendly kiosk interface.	Children

3.5 Non-Functional Requirements

6: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Offline Operation	The system shall operate entirely on the home local network with no internet dependency.
NFR-02	Performance	Screens shall respond within two seconds on a typical home tablet and server.
NFR-03	Usability	The child interface shall be usable by children aged 4–12 without text-heavy instructions.
NFR-04	Security	Parent functions shall require authentication; child data shall never leave the home.
NFR-05	Reliability	The system shall remain available while the home server and tablet are powered on.
NFR-06	Data Integrity	Coin balances must always match the sum of ledger transactions.
NFR-07	Maintainability	The modular architecture (presentation, API, services, data) shall allow independent changes.
NFR-08	Portability	The Flutter client shall run on Android and iOS tablets.
NFR-09	Scalability	The design shall comfortably support a family of up to eight children.
NFR-10	Backup	The parent shall be able to back up the database to a local file.
NFR-11	Compatibility	The API shall use standard REST/JSON over HTTP on the local network.

3.6 Business Rules

7: Business Rules

ID	Business Rule
BR-01	A child cannot be deleted if the child has verified assignments; the parent must deactivate the child instead.
BR-02	A task can only be created with a valid schedule type and a coin value greater than zero.
BR-03	Self-check tasks are verified immediately and cannot be rejected later.
BR-04	A parent-approval task can only be approved or rejected while its status is “pending verification”.
BR-05	Rejection always requires a reason and always reverses the temporary coins.
BR-06	A reward redemption is only possible if the child’s current balance is at least the reward cost.
BR-07	Coins can only be deducted through rejection reversal, parent removal or approved redemption.
BR-08	A child earns each badge only once unless the parent explicitly awards it again.
BR-09	A missed task cannot be completed after the due date unless the parent re-activates it.
BR-10	Report periods are computed from local dates: daily (1 day), weekly (Mon–Sun), monthly (1st–last), yearly (Jan–Dec).

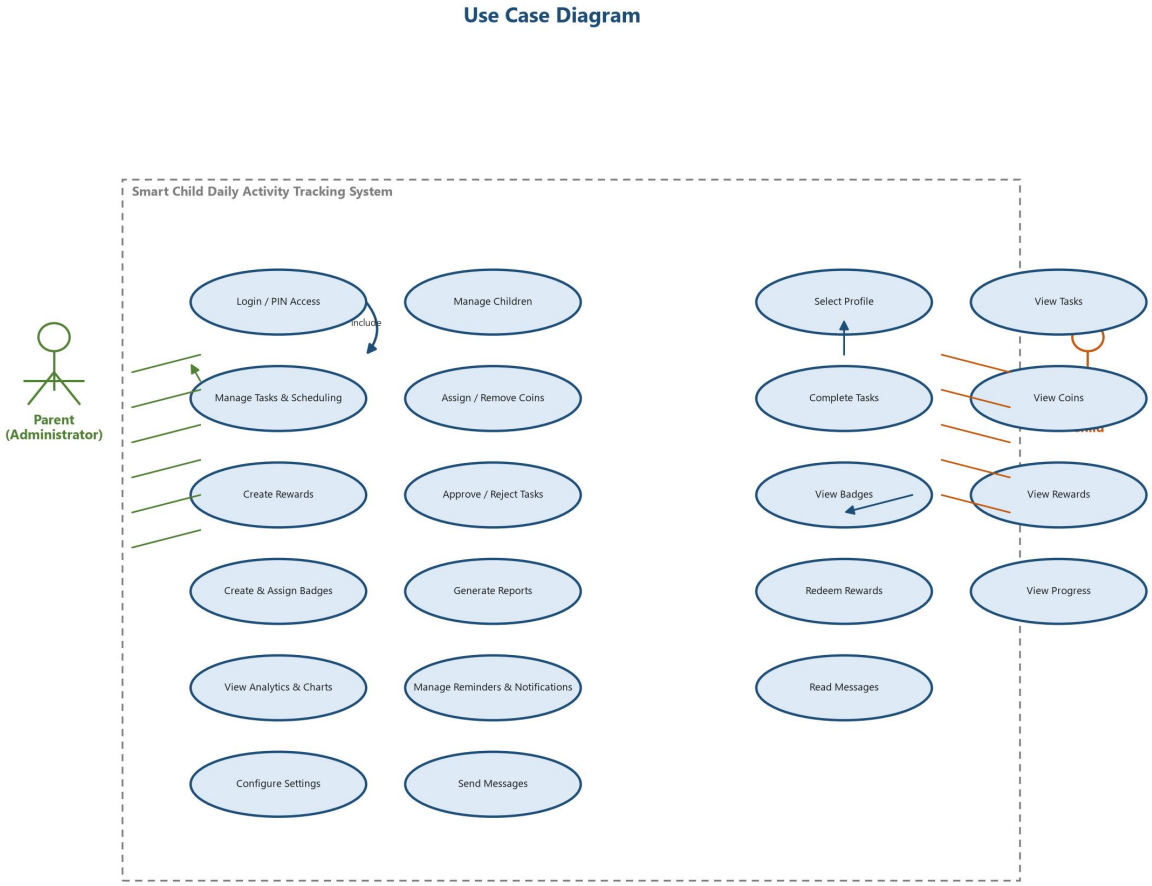
3.7 Actors

8: Actors of the System

Actor	Role	Goals
Parent (Administrator)	Owns and administers the entire system; the only actor with write access to configuration.	Login securely; manage children, tasks, coins, rewards, badges; approve or reject tasks; review reports and settings.
Child	Non-administrator end-user of the shared tablet; selects a profile and interacts with the kiosk interface.	View and complete tasks; earn coins; view badges, rewards and progress; read messages.
System Clock	Internal actor that generates assignments and marks missed tasks automatically.	Create assignments on scheduled dates and mark overdue tasks missed.

3.8 Use Case Modelling

3.8.1 Use Case Diagram



1*: Use Case Diagram of the System*

The use case diagram (Figure 1) shows the two human actors and the internal system-clock actor together with the main use cases of the system. The parent interacts with all administrative use cases, while the child interacts with the kiosk use cases. The dashed "include" relationship shows that coin-awarding behaviour is shared by the task-completion and approval use cases.

3.8.2 Use Case Descriptions

The most important use cases are described in detail below. Each description identifies the actor, preconditions, trigger, main flow, alternate flow, post-conditions and priority.

Field	Description
Use Case ID	UC-01
Use Case Name	Secure Login (Parent)
Actor(s)	Parent
Preconditions	The tablet is running and the system clock is

	synchronised.
Trigger	The parent opens the application.
Main Flow	1. System displays the login screen. 2. Parent enters username and password. 3. System validates credentials. 4. Parent enters the four-digit PIN. 5. System validates the PIN and opens the parent dashboard.
Alternate Flow	A1: Invalid credentials — system shows an error and logs the failed attempt. A2: Forgotten PIN — parent resets the PIN using the security answer.
Post-conditions	Parent session starts; administrator functions are unlocked.
Priority	High

Field	Description
Use Case ID	UC-02
Use Case Name	Manage Children
Actor(s)	Parent
Preconditions	Parent is logged in.
Trigger	Parent selects the Children option from the menu.
Main Flow	1. System lists existing children. 2. Parent chooses Add, Edit or Delete. 3. Parent enters name, avatar and date of birth. 4. System saves the child profile.
Alternate Flow	A1: Editing — updates are saved and logged. A2: Deleting a child with verified records — system blocks deletion and suggests deactivation (BR-01).
Post-conditions	Child profile is created, updated or deactivated.
Priority	High

Field	Description
Use Case ID	UC-03
Use Case Name	Create Task with Schedule
Actor(s)	Parent
Preconditions	Parent is logged in.
Trigger	Parent selects the Tasks option and presses Add Task.

Main Flow	1. System shows the task form. 2. Parent enters title and description. 3. Parent selects schedule type (daily, weekly, monthly, custom, weekend, specific days). 4. Parent sets repeat days and coin value. 5. Parent selects verification type (self-check or parent approval). 6. System saves the task.
Alternate Flow	A1: Invalid schedule or zero coins — system rejects with a message (BR-02).
Post-conditions	Task is stored and assignments are generated for the relevant dates (FR-08).
Priority	High

Field	Description
Use Case ID	UC-04
Use Case Name	Complete Task (Self-Check)
Actor(s)	Child
Preconditions	The child has selected a profile and the task is available for today.
Trigger	The child presses the Complete button on a self-check task.
Main Flow	1. System asks for confirmation. 2. Child confirms. 3. System credits coins immediately. 4. System sets status to “verified”. 5. System records a ledger transaction and updates the balance.
Alternate Flow	A1: Task already completed — system prevents double completion (BR-04).
Post-conditions	Task verified; balance increased by the task coin value.
Priority	High

Field	Description
Use Case ID	UC-05
Use Case Name	Complete Task (Parent Approval) and Approve / Reject
Actor(s)	Child, Parent
Preconditions	The child has selected a profile; the task requires

	parent approval.
Trigger	The child submits the task for verification.
Main Flow	1. Child submits the task. 2. System credits temporary coins and sets status to “pending verification”. 3. Parent reviews the task in the dashboard. 4. Parent approves — system finalises coins and sets status to “verified”. 5. Parent rejects — system selects a reason, removes the temporary coins and sets status to “rejected”.
Alternate Flow	A1: Rejection — coins are automatically deducted and recorded (FR-13). A2: No action — task remains pending until the parent acts.
Post-conditions	Task is either verified with final coins or rejected with a reason and reversed coins.
Priority	High

Field	Description
Use Case ID	UC-06
Use Case Name	Redeem Reward
Actor(s)	Child, Parent
Preconditions	The child has a profile and at least one active reward exists.
Trigger	The child selects a reward and presses Redeem.
Main Flow	1. System checks the child’s balance against the reward cost. 2. If sufficient, system creates a redemption request with status “pending”. 3. Parent approves the request. 4. System deducts the coins, grants the reward and notifies the child.
Alternate Flow	A1: Insufficient balance — system shows “not enough coins” (BR-06). A2: Parent rejects — request is cancelled with no deduction.
Post-conditions	Redemption completed or cancelled; ledger reflects any deduction.
Priority	Medium

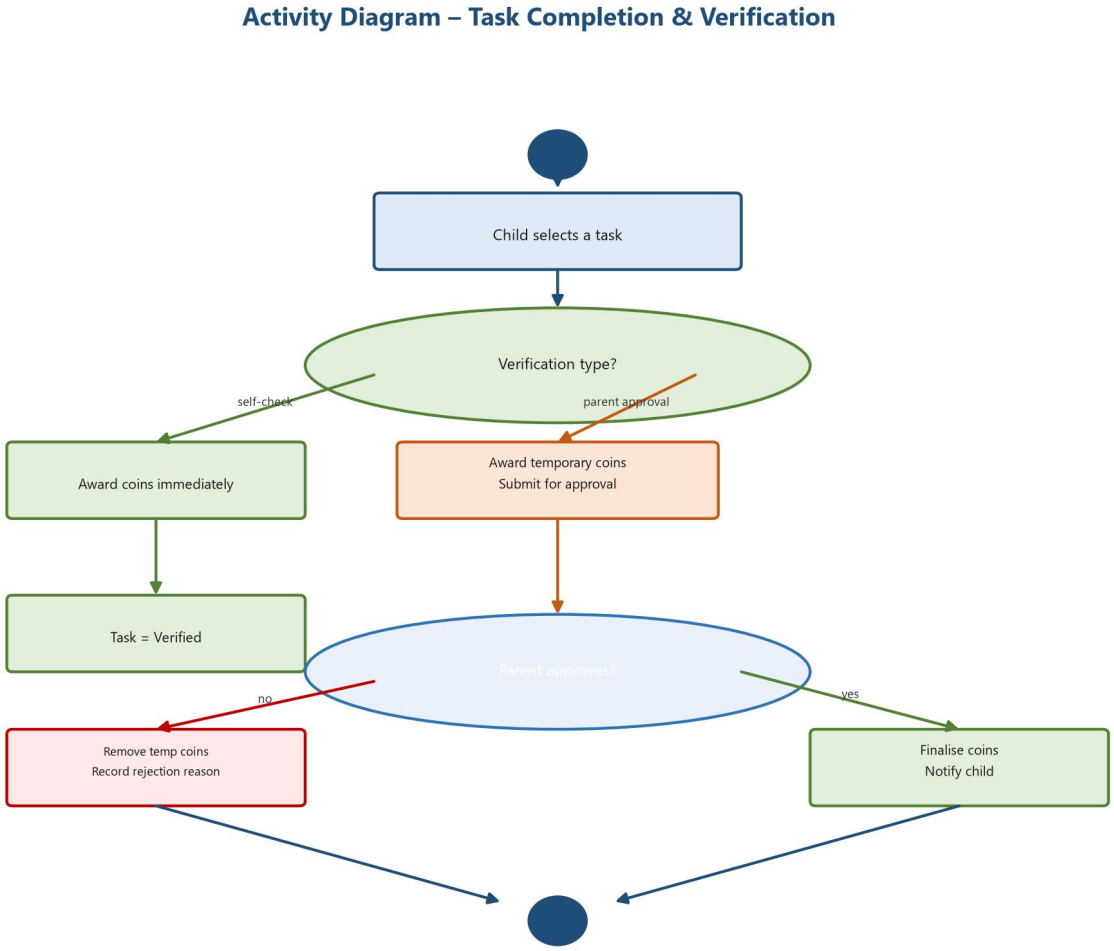
Field	Description

Use Case ID	UC-07
Use Case Name	Generate Reports
Actor(s)	Parent
Preconditions	Parent is logged in and child data exists.
Trigger	Parent opens the Reports screen and selects a period.
Main Flow	1. Parent selects period (daily, weekly, monthly, yearly) and optionally a child. 2. System computes completed, rejected, pending and missed tasks, coins, badges, completion rate and top child. 3. System displays charts and tables. 4. Parent may print or export the report.
Alternate Flow	A1: No data for the period — system displays an empty-state message.
Post-conditions	Report generated and displayed.
Priority	High

Field	Description
Use Case ID	UC-08
Use Case Name	Assign Badges and Bonus Coins
Actor(s)	Parent
Preconditions	Parent is logged in and at least one child exists.
Trigger	Parent selects a child and chooses Assign Badge or Bonus Coins.
Main Flow	1. Parent selects the badge from the library or creates a new one. 2. Parent assigns the badge to the child. 3. Alternatively, parent enters a bonus amount and a reason. 4. System records the award or credit and notifies the child.
Alternate Flow	A1: Duplicate badge — system warns unless explicitly re-awarded (BR-08).
Post-conditions	Badge awarded or coins credited with a ledger record.
Priority	Medium

3.8.3 Activity Diagram

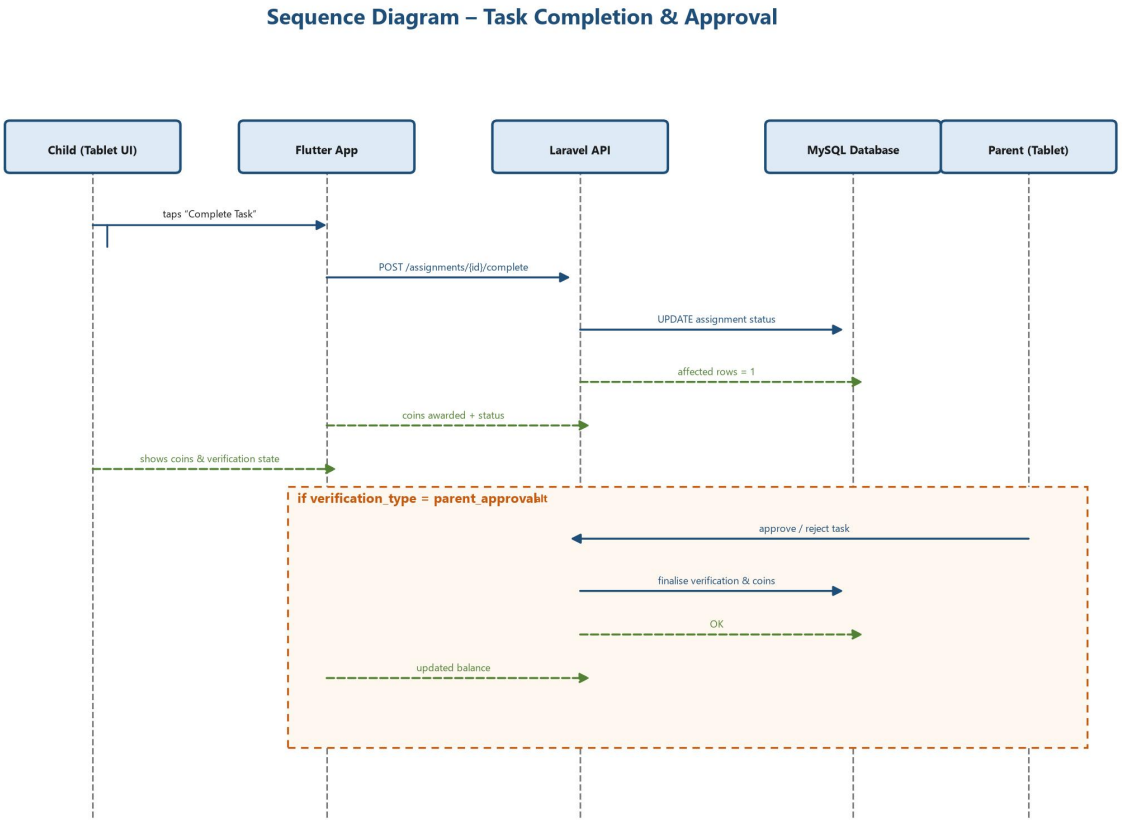
The activity diagram below models the task-completion and verification process. It shows the decision on verification type, the immediate awarding of coins for self-check tasks, and the approval loop with automatic coin reversal on rejection for parent-approval tasks.



2: Activity Diagram – Task Completion and Verification

3.8.4 Sequence Diagram

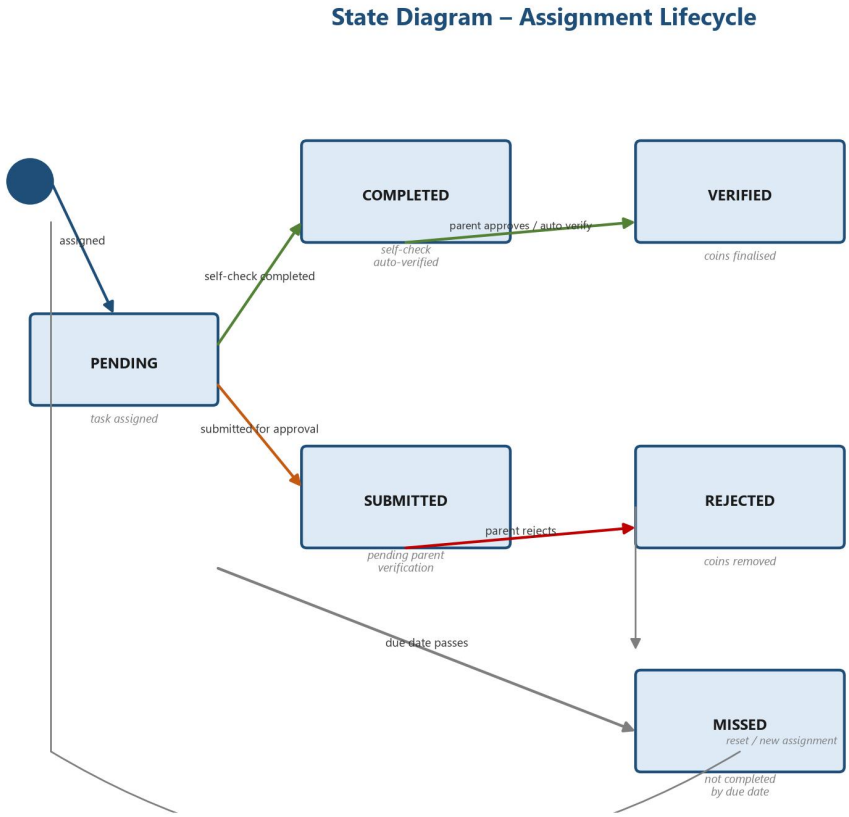
The sequence diagram below illustrates the interactions between the child-facing tablet UI, the Flutter application, the Laravel REST API and the MySQL database during task completion and parental verification. All calls use JSON over HTTP on the local network.



3*: Sequence Diagram – Task Completion and Parent Approval*

3.8.5 State Diagram

The state diagram below models the lifecycle of an assignment (a single task instance). A task may be pending, completed, submitted, verified, rejected or missed, with well-defined transitions triggered by child actions, parent decisions or the system clock.

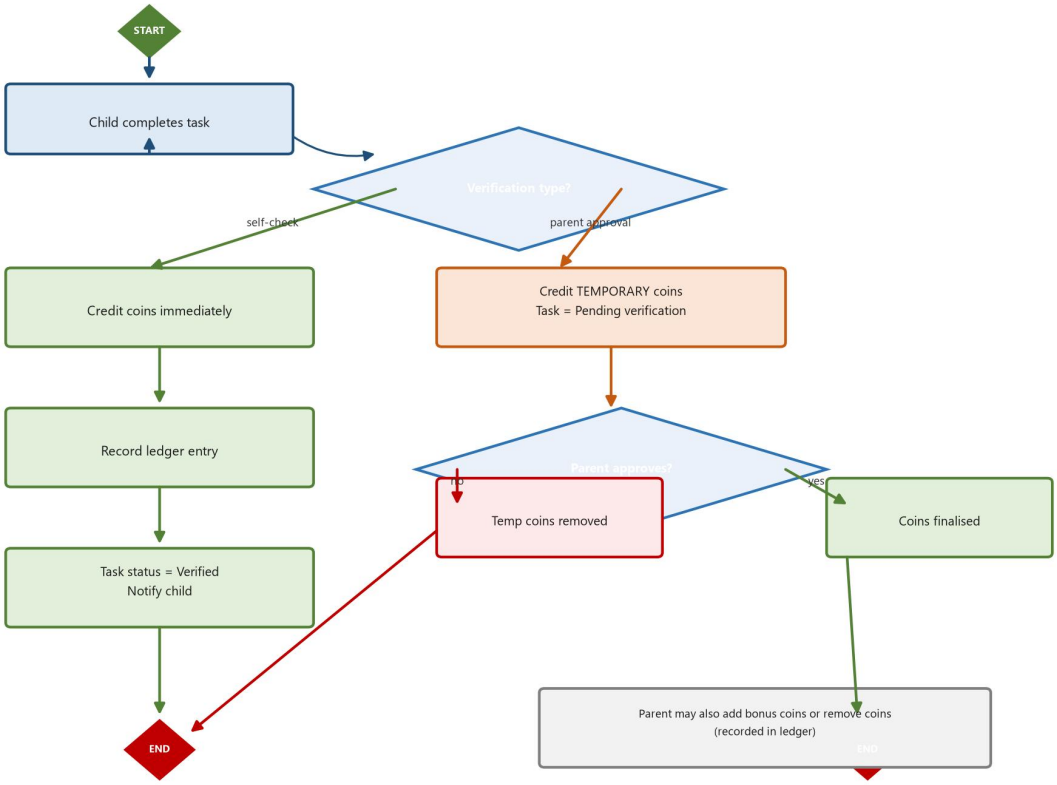


4*: State Diagram – Assignment Lifecycle*

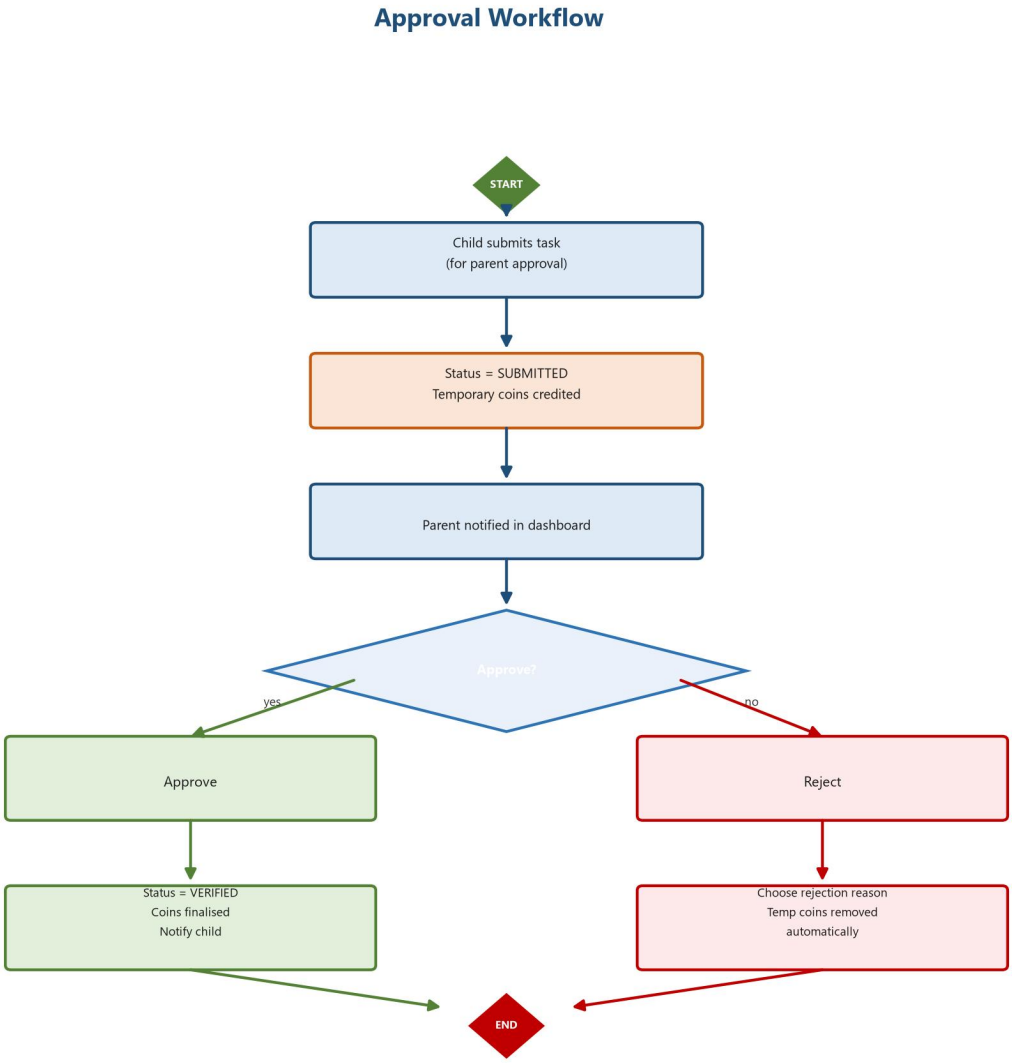
3.8.6 Workflow Diagrams

The following workflow diagrams detail the coin, approval, reward, parent and child processes that define the behaviour of the system.

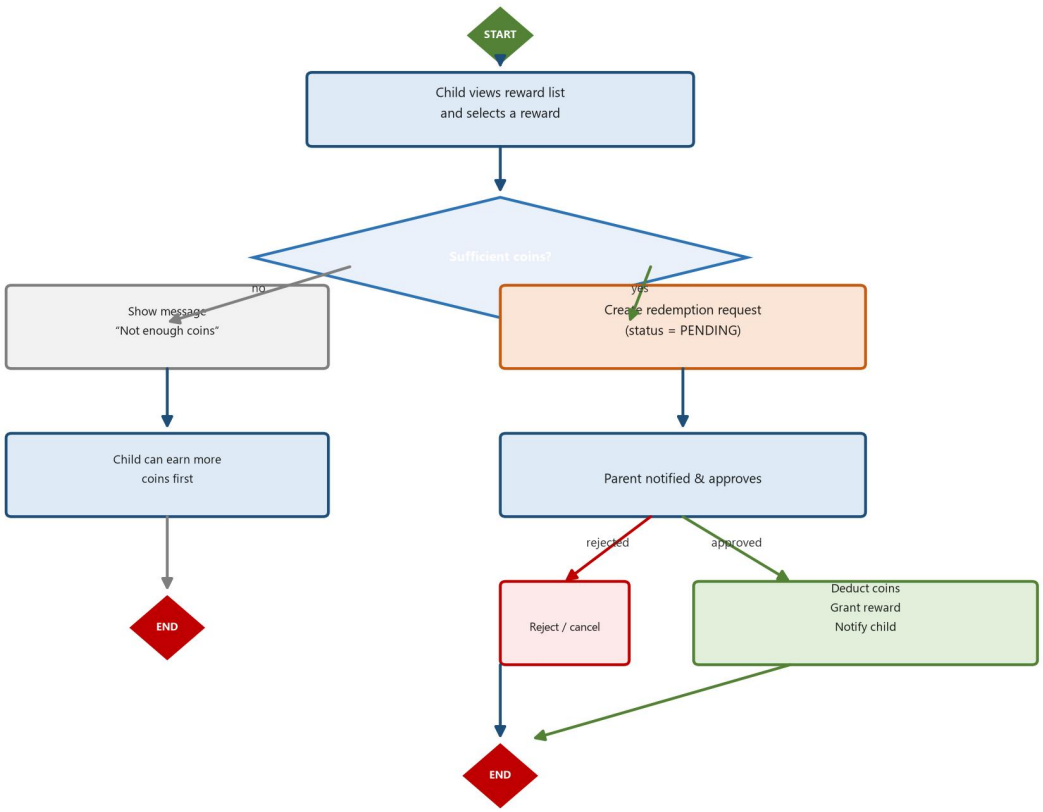
Coin Workflow



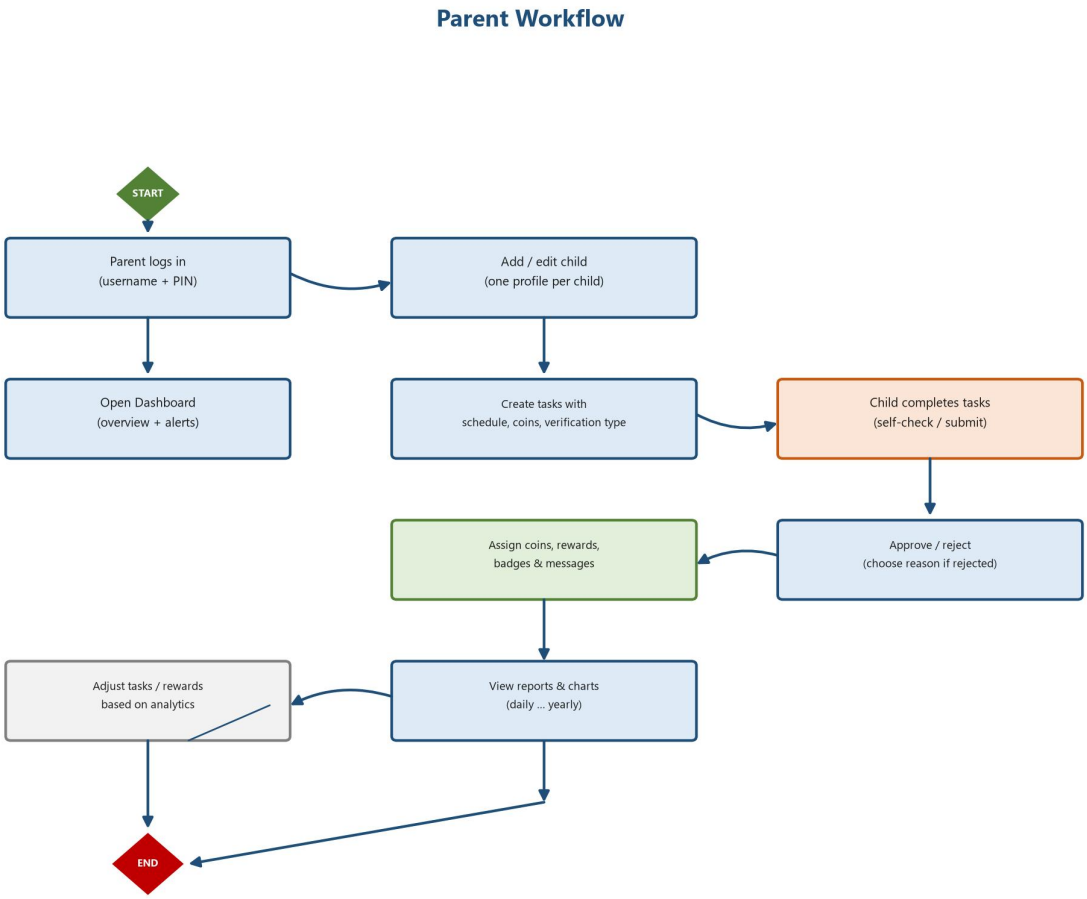
5: Coin Workflow



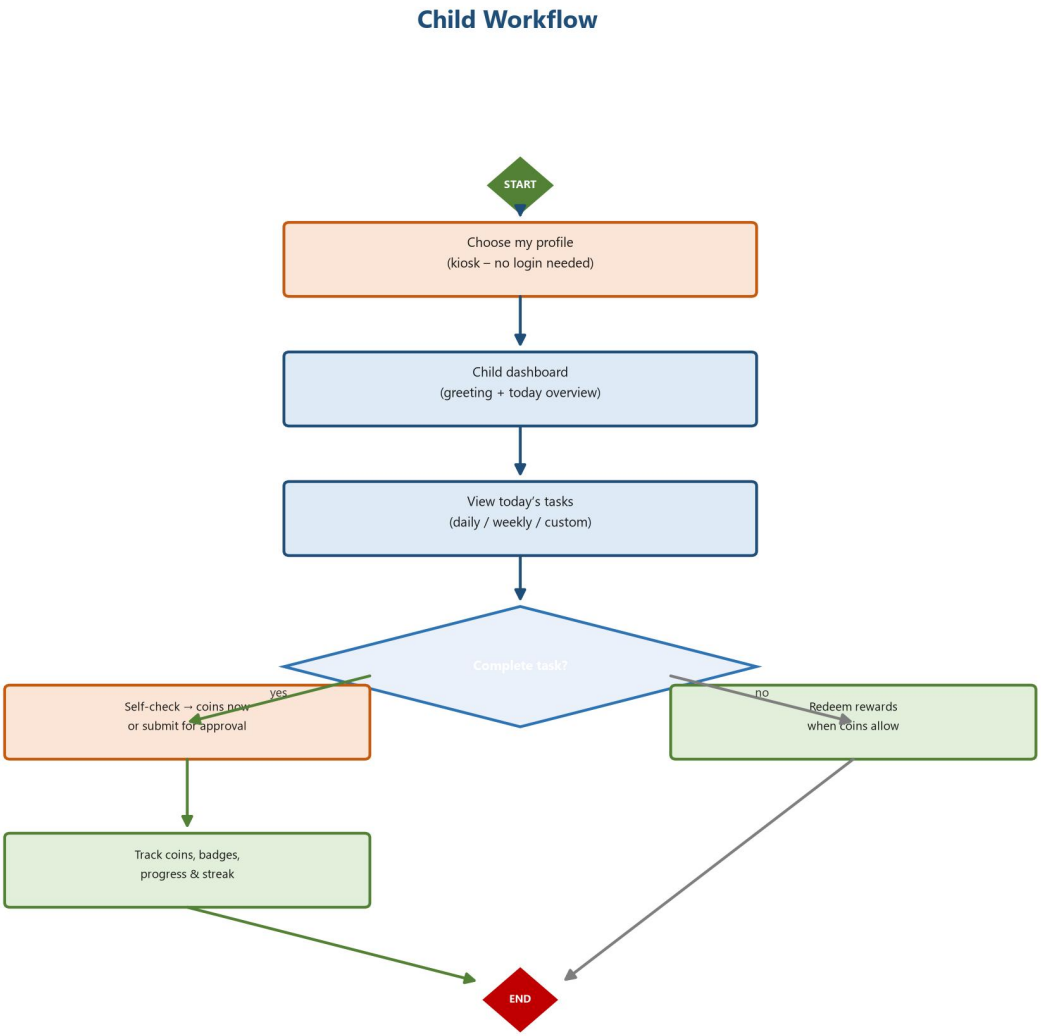
6: *Approval Workflow*



7: Reward Workflow



8: *Parent Workflow*

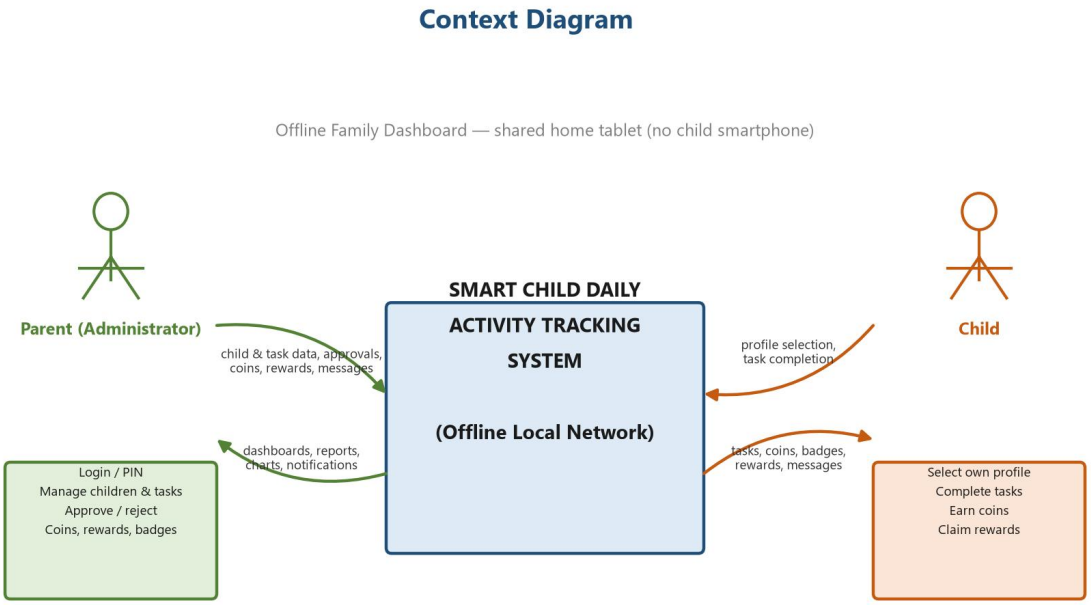


9: Child Workflow

3.9 Data Flow Analysis

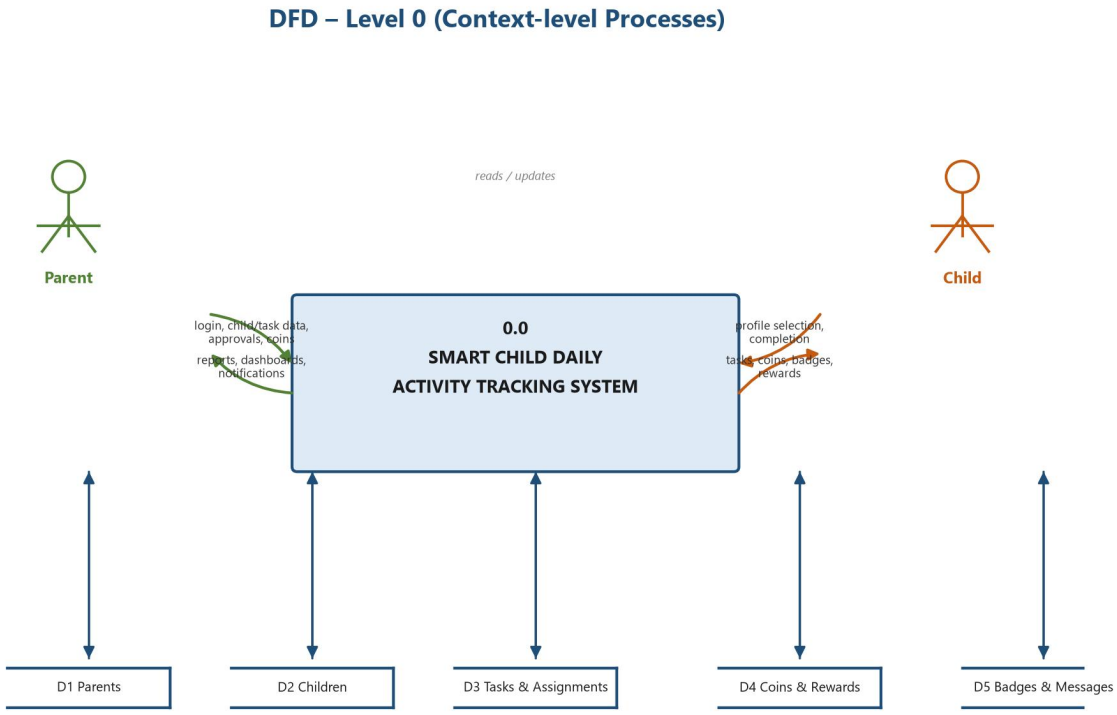
3.9.1 Context Diagram

The context diagram below shows the entire system as a single process together with its two external entities — the parent and the child — and the main data flows between them. No data flows leave the home network.



10: *Context Diagram of the System*

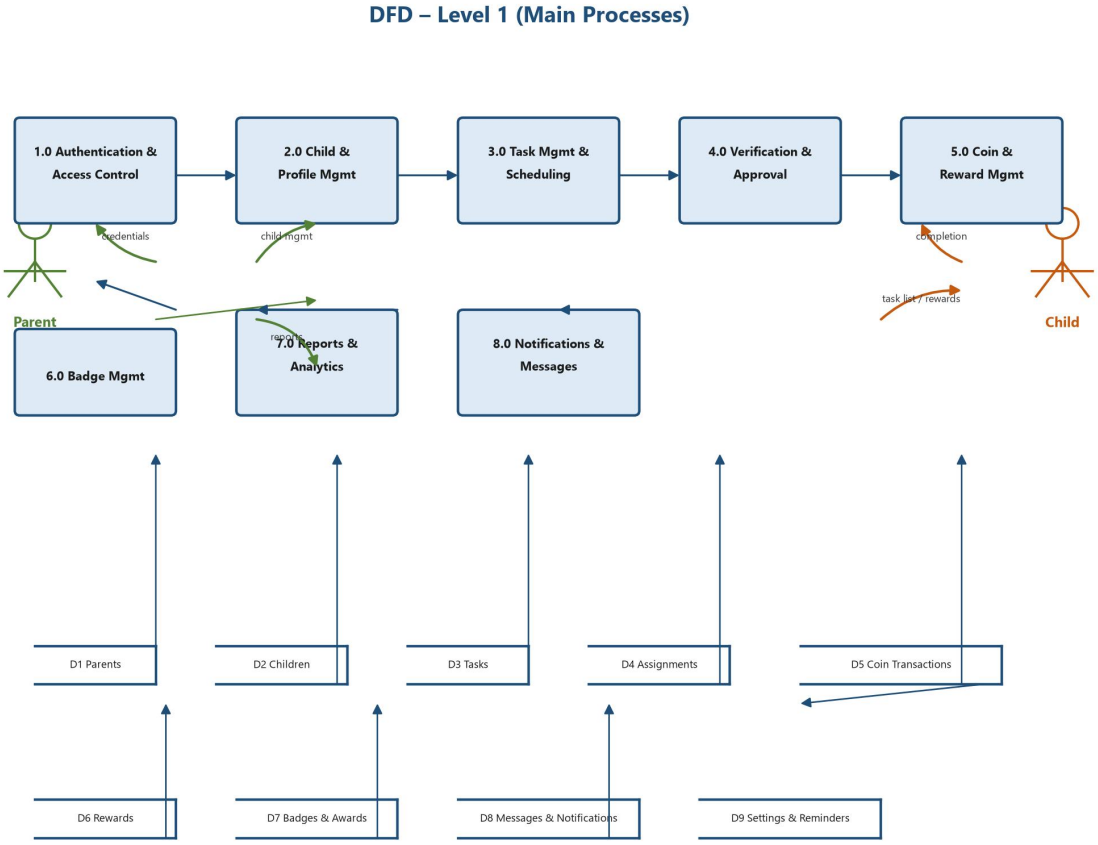
3.9.2 Data Flow Diagram – Level 0



11: *DFD Level 0*

The level-0 diagram (Figure 11) decomposes the context diagram into the main system process and its principal data stores. The system exchanges management inputs and reports with the parent, and task, coin and reward data with the child, while reading and updating its stores.

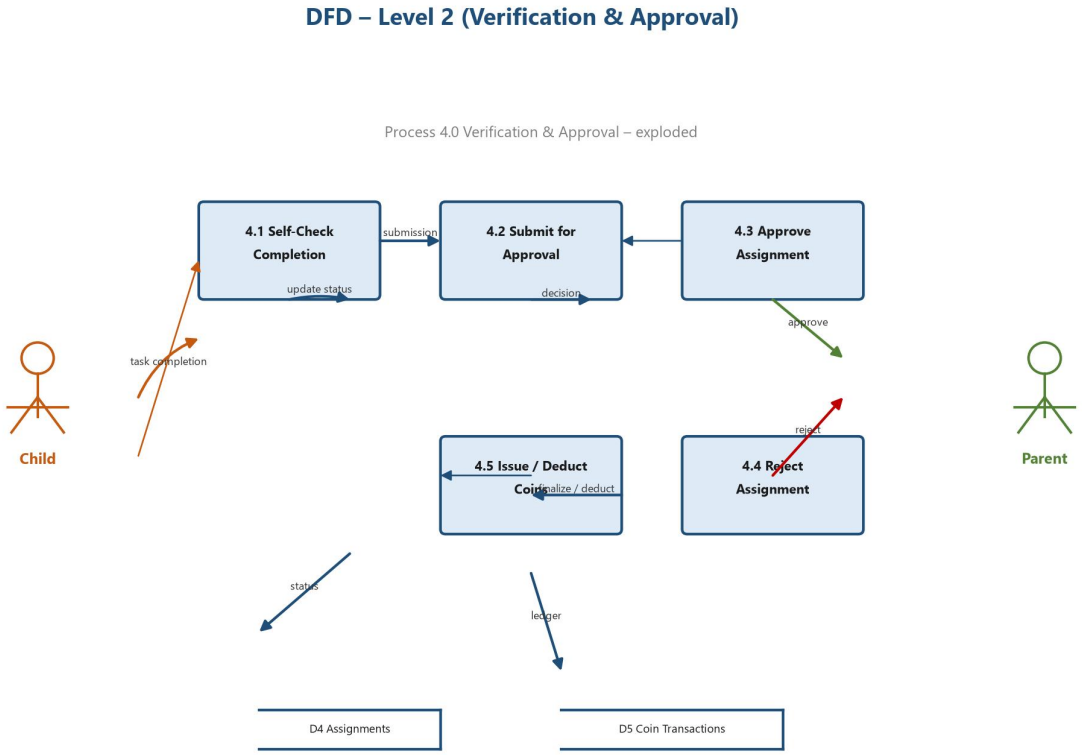
3.9.3 Data Flow Diagram – Level 1



12*: DFD Level 1 – Main Processes*

The level-1 diagram (Figure 12) expands the system into eight major processes: authentication, child and profile management, task management and scheduling, verification and approval, coin and reward management, badge management, reports and analytics, and notifications and messages. Each process reads from and writes to the shared data stores.

3.9.4 Data Flow Diagram – Level 2

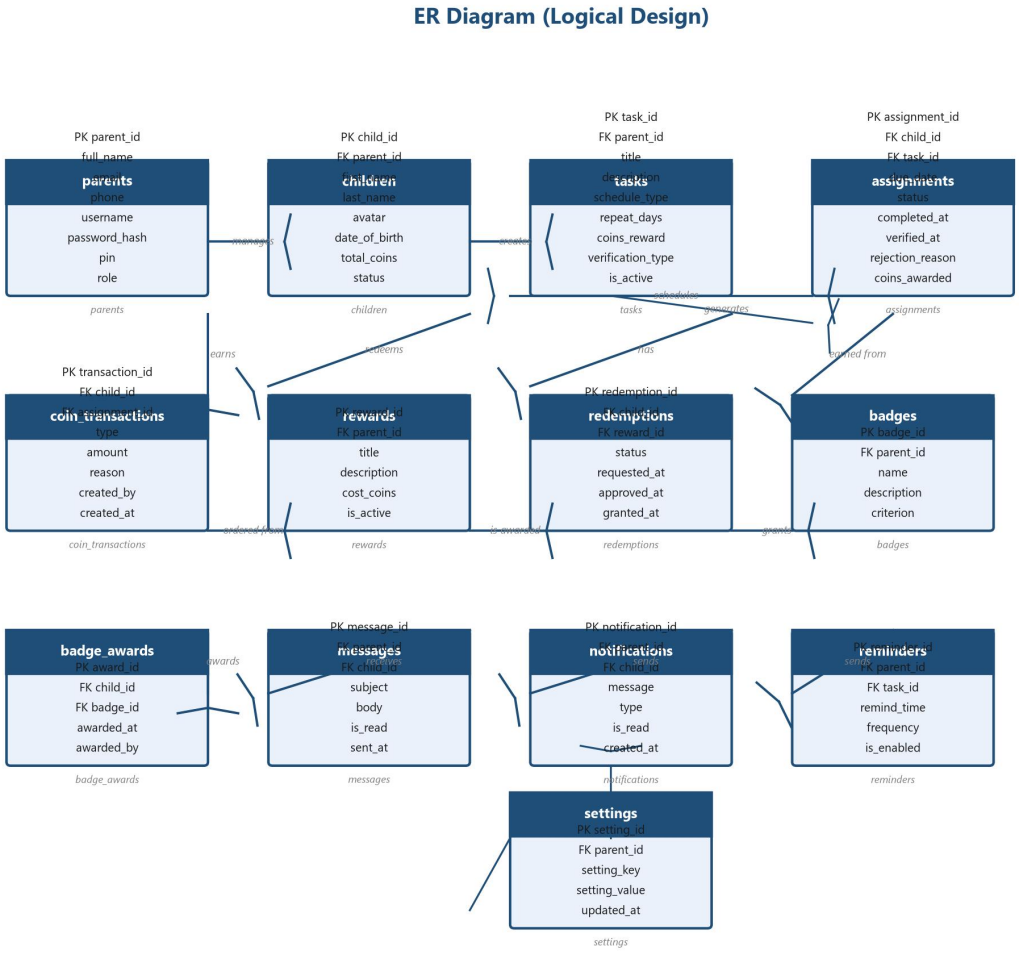


13: *DFD Level 2 – Verification and Approval*

The level-2 diagram (Figure 13) explodes the verification and approval process (Process 4.0) to show the detailed flows for self-check completion, submission, approval, rejection and coin issuing or deduction.

3.10 Data Design

3.10.1 Entity-Relationship Diagram



14*: Entity-Relationship Diagram (Logical Design)*

The ER diagram (Figure 14) models the data of the system in third normal form. Thirteen entities capture parents, children, tasks, assignments, coin transactions, rewards, redemptions, badges, badge awards, messages, notifications, reminders and settings, together with their relationships and cardinalities.

3.10.2 Database Design and Data Dictionary

The logical design is implemented in MySQL as the relational schema described below. Each entity becomes a table with a primary key, and relationships become foreign keys. The data dictionary lists every column with its data type, key role and description.

9*: Data Dictionary of the MySQL Schema*

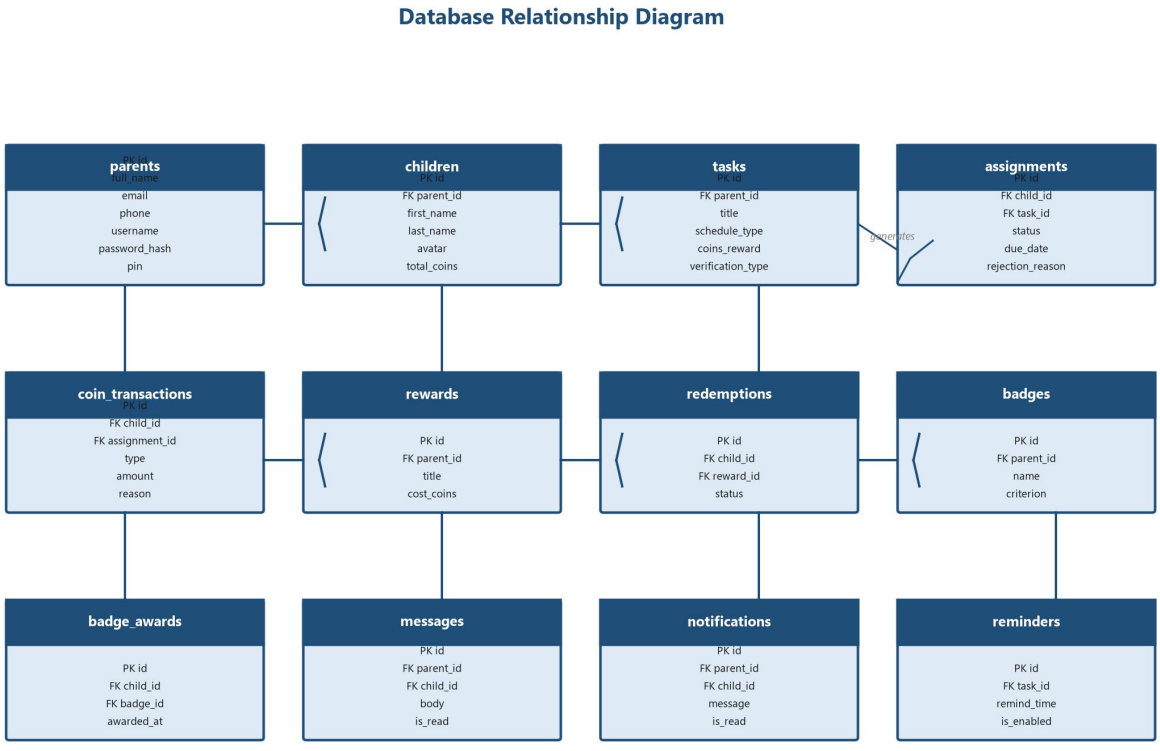
Table	Column	Data Type	Key	Description
parents	parent_id	INT	PK	Unique parent identifier
parents	full_name	VARCHAR(100)	–	Parent full name

parents	email	VARCHAR(120)	–	Contact email (optional)
parents	phone	VARCHAR(20)	–	Contact phone (optional)
parents	username	VARCHAR(50)	UQ	Login username
parents	password_hash	VARCHAR(255)	–	Hashed password
parents	pin	VARCHAR(255)	–	Hashed four-digit PIN
parents	role	ENUM	–	Administrator role
children	child_id	INT	PK	Unique child identifier
children	parent_id	INT	FK	Owning parent
children	first_name	VARCHAR(50)	–	Child first name
children	last_name	VARCHAR(50)	–	Child last name
children	avatar	VARCHAR(255)	–	Avatar image path
children	date_of_birth	DATE	–	Date of birth
children	total_coins	INT	–	Current coin balance
children	status	ENUM	–	Active / inactive
tasks	task_id	INT	PK	Unique task identifier
tasks	parent_id	INT	FK	Creating parent
tasks	title	VARCHAR(100)	–	Task title
tasks	description	TEXT	–	Task description
tasks	schedule_type	ENUM	–	Daily / weekly / monthly / custom / weekend / specific days
tasks	repeat_days	VARCHAR(50)	–	Days of week for repeating schedules
tasks	coins_reward	INT	–	Coins earned on completion
tasks	verification_type	ENUM	–	Self-check / parent approval
tasks	is_active	BOOLEAN	–	Task enabled or not
assignments	assignment_id	INT	PK	Unique assignment identifier
assignments	child_id	INT	FK	Assigned child
assignments	task_id	INT	FK	Source task
assignments	due_date	DATE	–	Scheduled date
assignments	status	ENUM	–	Pending / completed / submitted / verified / rejected / missed
assignments	completed_at	DATETIME	–	Completion timestamp
assignments	verified_at	DATETIME	–	Verification timestamp
assignments	rejection_reason	VARCHAR(255)	–	Reason when rejected
assignments	coins_awarded	INT	–	Final coins for this assignment
coin_transactions	transaction_id	INT	PK	Unique transaction identifier
coin_transactions	child_id	INT	FK	Affected child
coin_transactions	assignment_id	INT	FK	Related assignment (nullable)

coin_transactions	type	ENUM	–	Earn / bonus / redeem / deduction
coin_transactions	amount	INT	–	Signed coin amount
coin_transactions	reason	VARCHAR(255)	–	Human-readable reason
coin_transactions	created_by	VARCHAR(20)	–	parent / system
coin_transactions	created_at	DATETIME	–	Ledger timestamp
rewards	reward_id	INT	PK	Unique reward identifier
rewards	parent_id	INT	FK	Creating parent
rewards	title	VARCHAR(100)	–	Reward name
rewards	description	TEXT	–	Reward description
rewards	cost_coins	INT	–	Coins required
rewards	is_active	BOOLEAN	–	Available or not
redemptions	redemption_id	INT	PK	Unique redemption identifier
redemptions	child_id	INT	FK	Requesting child
redemptions	reward_id	INT	FK	Requested reward
redemptions	status	ENUM	–	Pending / approved / rejected / granted
redemptions	requested_at	DATETIME	–	Request time
redemptions	granted_at	DATETIME	–	Grant time
badges	badge_id	INT	PK	Unique badge identifier
badges	parent_id	INT	FK	Creating parent
badges	name	VARCHAR(100)	–	Badge name
badges	description	TEXT	–	Badge description
badges	criterion	VARCHAR(255)	–	Award criterion
badge_awards	award_id	INT	PK	Unique award identifier
badge_awards	child_id	INT	FK	Awarded child
badge_awards	badge_id	INT	FK	Awarded badge
badge_awards	awarded_at	DATETIME	–	Award time
badge_awards	awarded_by	VARCHAR(20)	–	Awarding actor
messages	message_id	INT	PK	Unique message identifier
messages	parent_id	INT	FK	Sender
messages	child_id	INT	FK	Recipient child
messages	subject	VARCHAR(120)	–	Message subject
messages	body	TEXT	–	Message body
messages	is_read	BOOLEAN	–	Read status
messages	sent_at	DATETIME	–	Sent time
notifications	notification_id	INT	PK	Unique notification identifier
notifications	parent_id	INT	FK	Recipient parent (nullable)
notifications	child_id	INT	FK	Recipient child (nullable)

notifications	message	VARCHAR(255)	–	Notification text
notifications	type	ENUM	–	Approval / coins / reward / badge / reminder
notifications	is_read	BOOLEAN	–	Read status
notifications	created_at	DATETIME	–	Creation time
reminders	reminder_id	INT	PK	Unique reminder identifier
reminders	parent_id	INT	FK	Creating parent
reminders	task_id	INT	FK	Related task
reminders	remind_time	TIME	–	Daily reminder time
reminders	frequency	VARCHAR(30)	–	Daily / specific days
reminders	is_enabled	BOOLEAN	–	Enabled or not
settings	setting_id	INT	PK	Unique setting identifier
settings	parent_id	INT	FK	Owning parent
settings	setting_key	VARCHAR(60)	–	Setting name
settings	setting_value	VARCHAR(255)	–	Setting value
settings	updated_at	DATETIME	–	Last update time

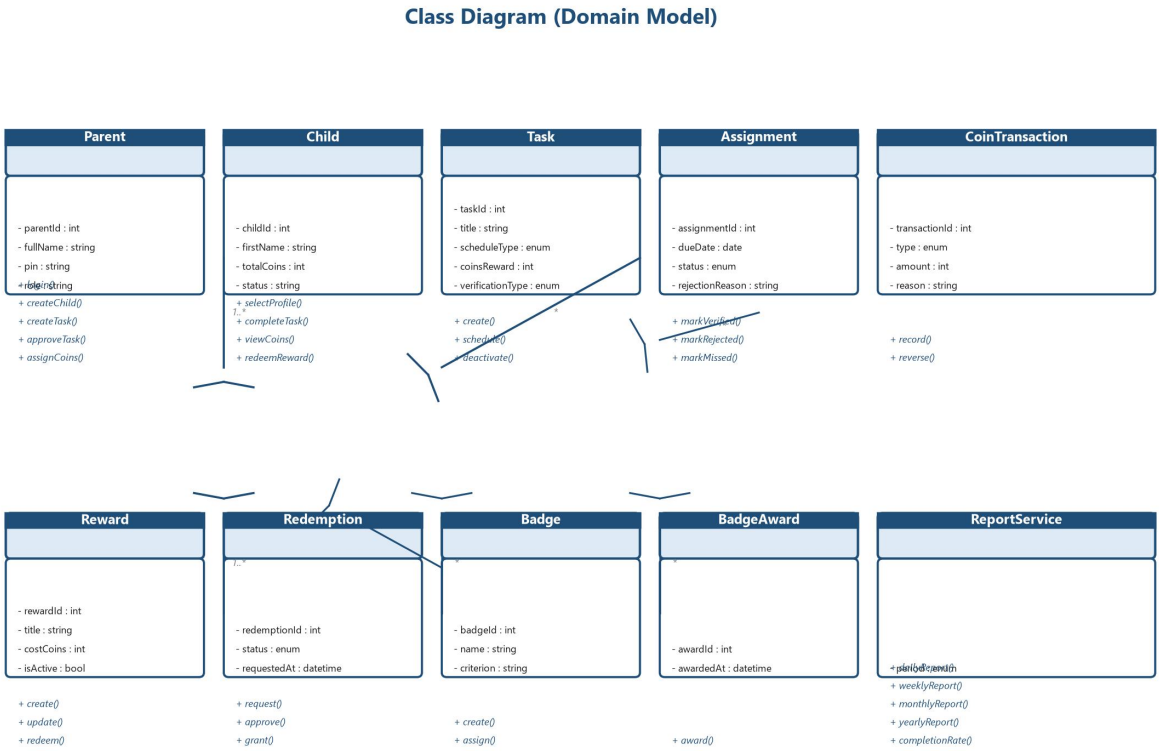
3.10.3 Database Relationship Diagram



15: *Database Relationship Diagram*

The database relationship diagram (Figure 15) shows the implemented tables with their primary and foreign keys. Referential integrity is enforced by foreign-key constraints so that orphaned records cannot occur.

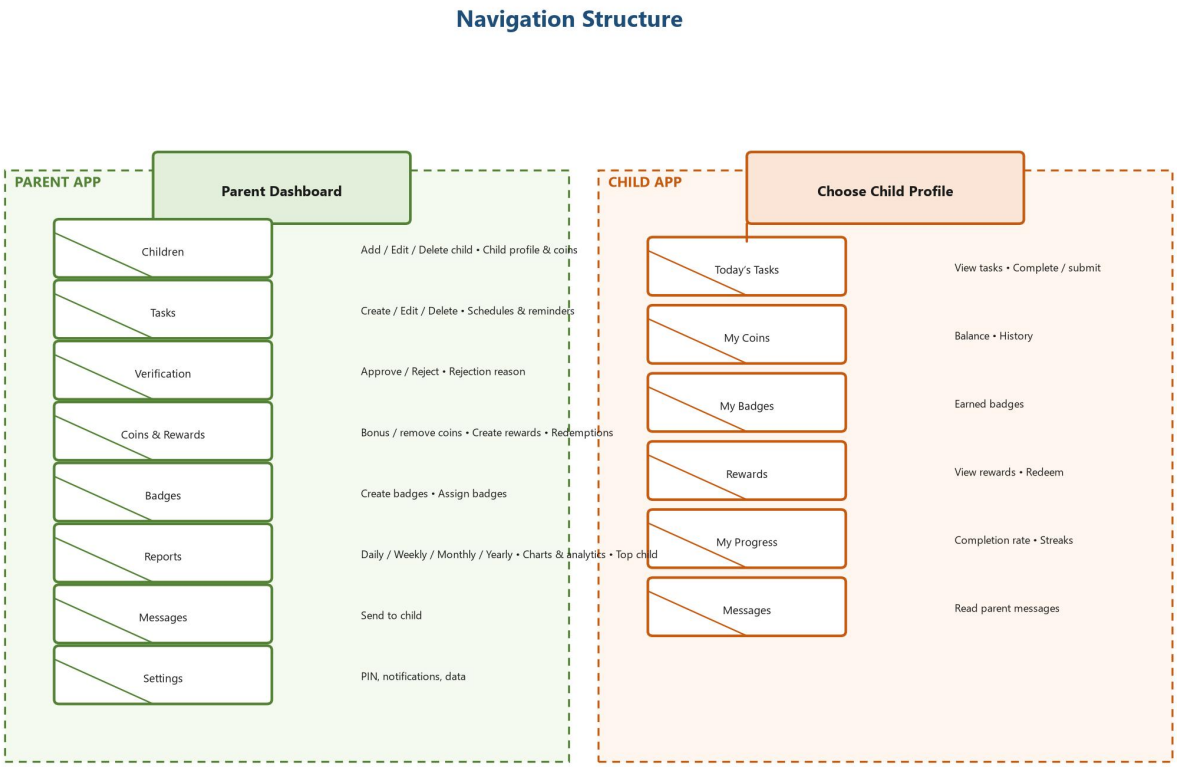
3.11 Class Diagram



16*: Class Diagram (Domain Model)*

The class diagram (Figure 16) presents the domain model of the system. It shows the principal classes, their attributes and operations, and the associations with multiplicity between parents, children, tasks, assignments, coins, rewards, redemptions, badges and the report service.

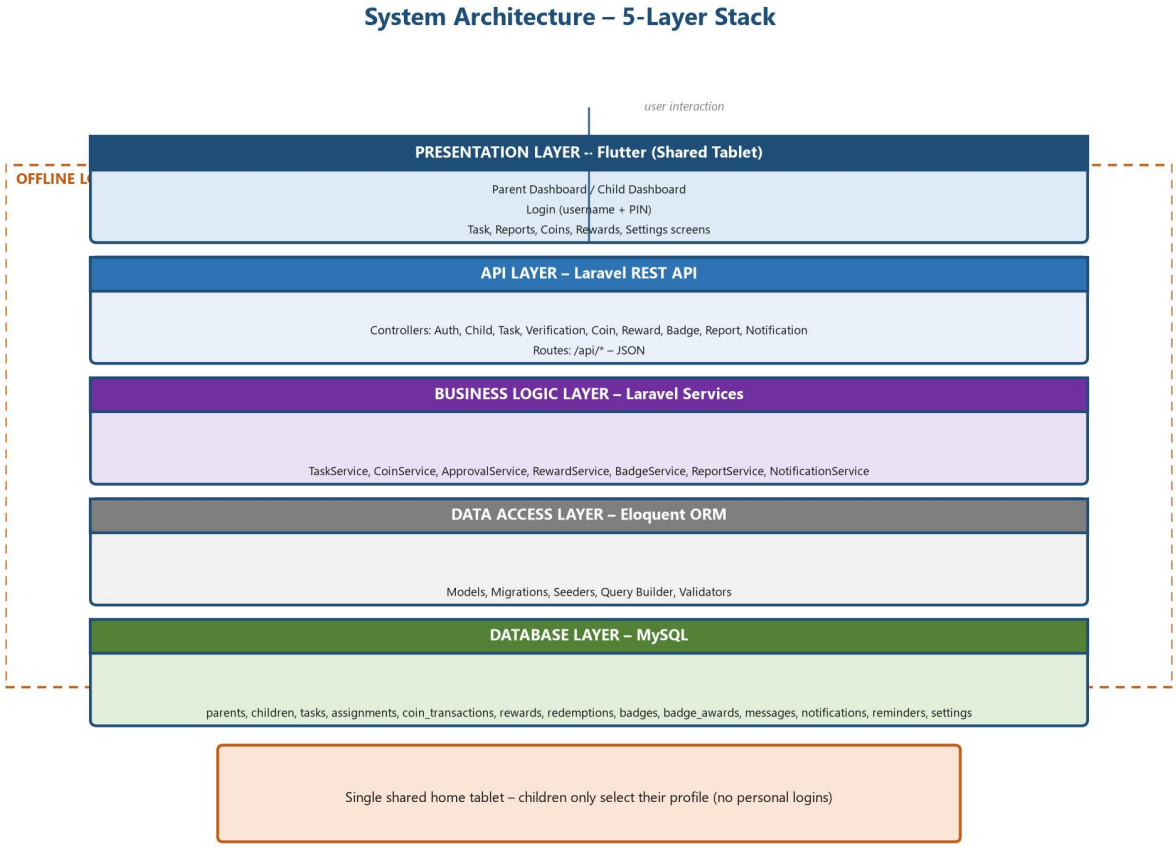
3.12 Navigation Structure



17*: Navigation Structure of the Application*

The navigation diagram (Figure 17) defines the screen structure of both the parent application and the child kiosk application. The parent flow starts at a secure login and branches through the administration modules, while the child flow starts at profile selection and branches through tasks, coins, badges, rewards, progress and messages.

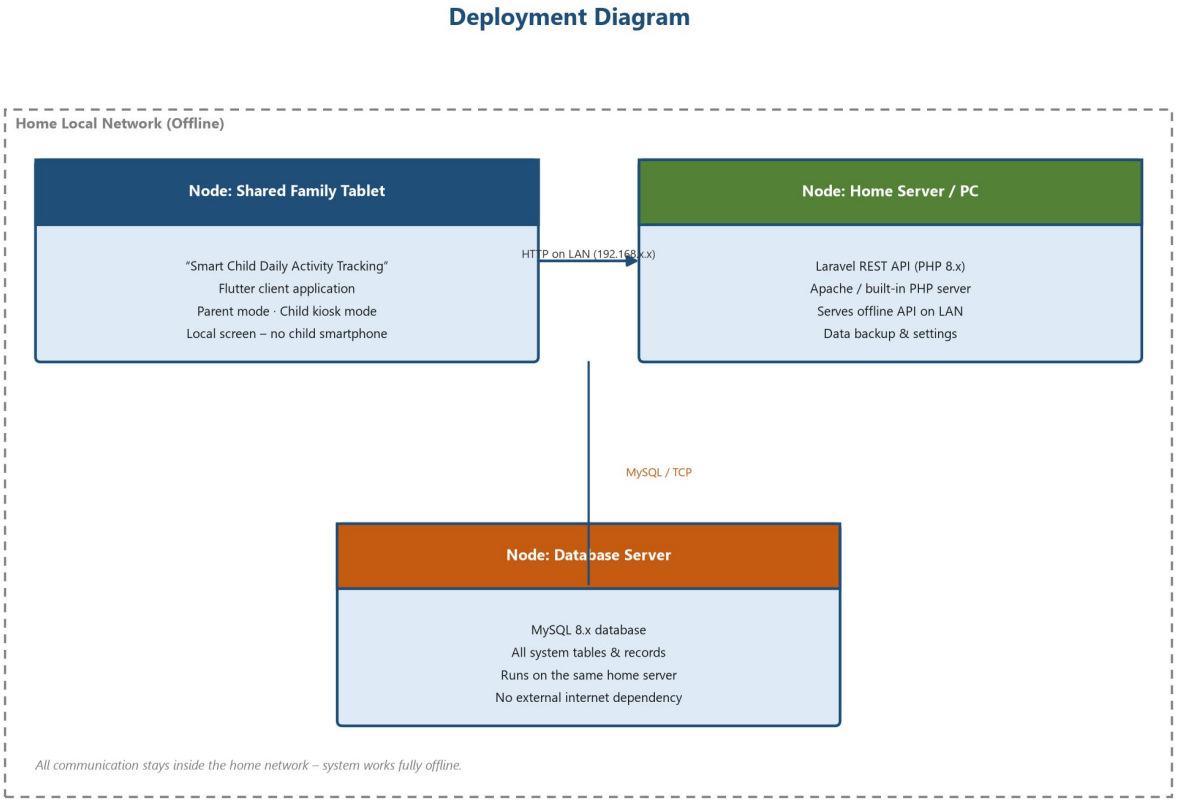
3.13 System Architecture



18: *System Architecture – Five-Layer Stack*

The system uses a five-layer architecture (Figure 18). The presentation layer is a Flutter client running on the shared tablet in either parent or child mode. The API layer exposes a Laravel REST API, the business-logic layer contains the service classes, the data-access layer uses Eloquent ORM, and the database layer is MySQL. All layers communicate over the offline local network.

3.14 Deployment Architecture



19: *Deployment Architecture*

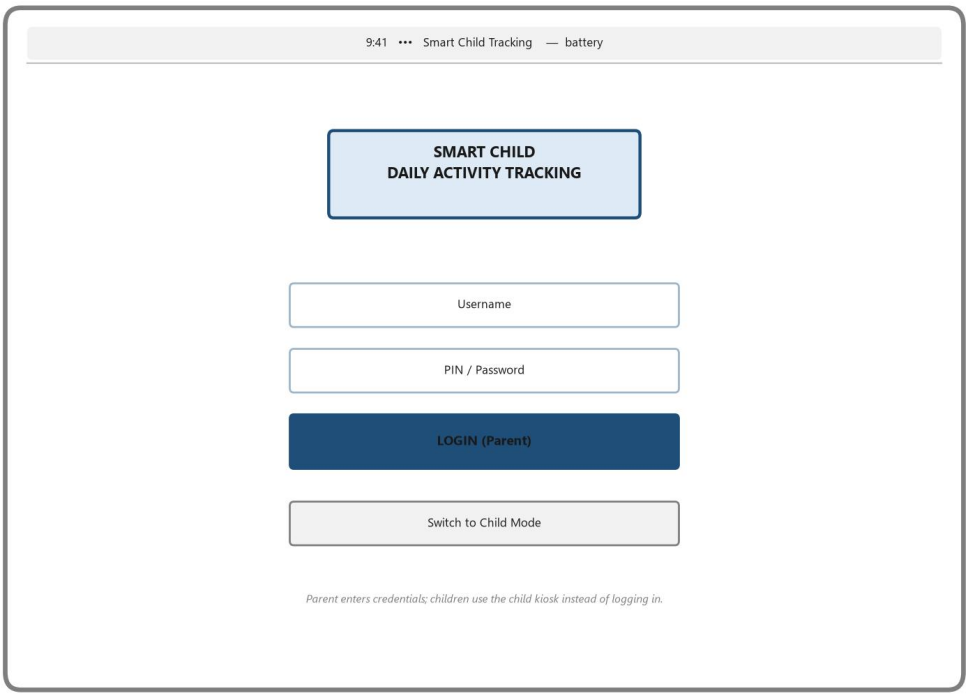
The deployment diagram (Figure 19) shows the physical nodes: the shared family tablet hosting the Flutter client, the home computer or server hosting the Laravel API and MySQL database, and the home Wi-Fi network that links them. There is no external internet dependency, and all data remains within the home.

3.15 User Interface Design (Wireframes)

3.15.1 Design Guidelines

The user interface follows simple, consistent guidelines. The parent interface uses a structured administration layout with a navigation menu, data cards, approval queues and charts. The child interface is a kiosk: large touch targets, avatars, bright colours, immediate feedback and no text-heavy screens. All screens are designed for a portrait tablet with at least a 10-inch screen. The wireframes below illustrate the visual structure of each screen.

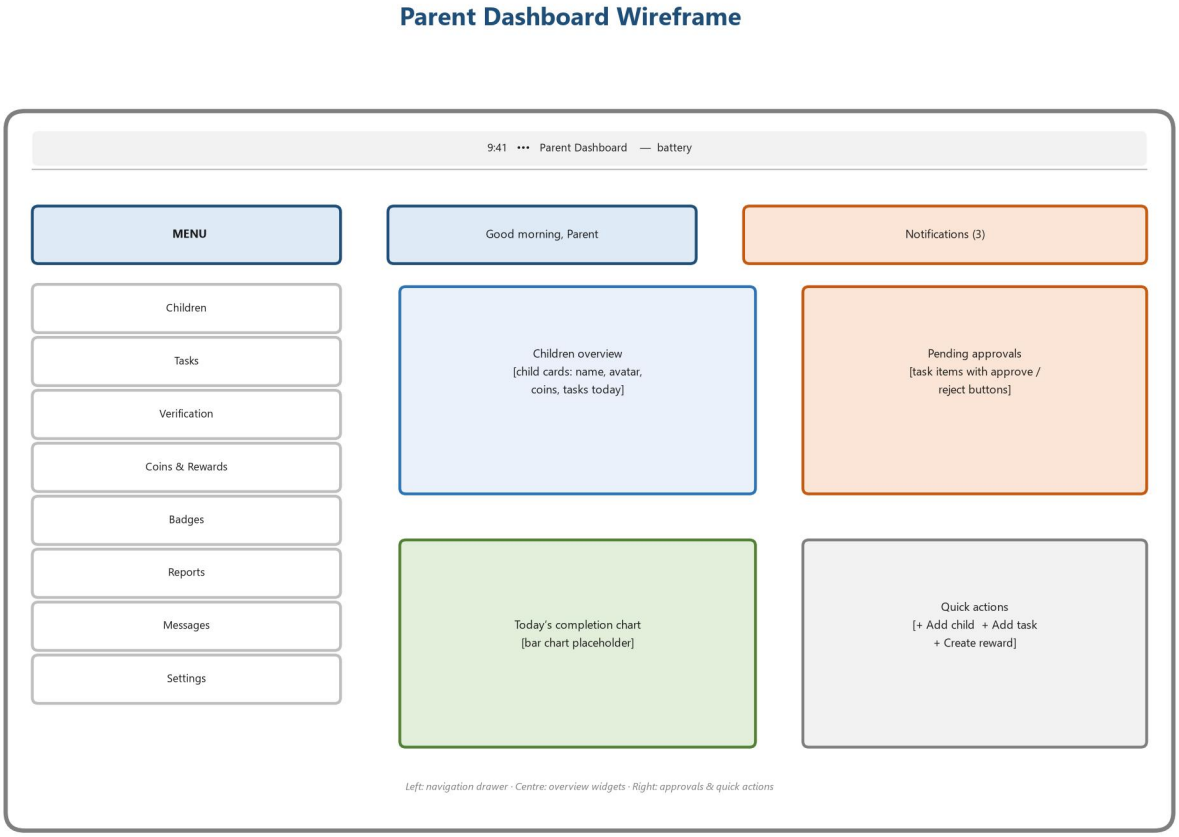
3.15.2 Login / PIN Screen



20*: Login / PIN Screen – Wireframe*

The login screen (Figure 20) is used only by the parent. The child-facing mode is entered through a separate kiosk button that leads to profile selection without any authentication.

3.15.3 Parent Dashboard



21: *Parent Dashboard – Wireframe*

The parent dashboard (Figure 21) combines a navigation drawer, an overview of children, a pending approval queue and quick actions.

3.15.4 Child Dashboard

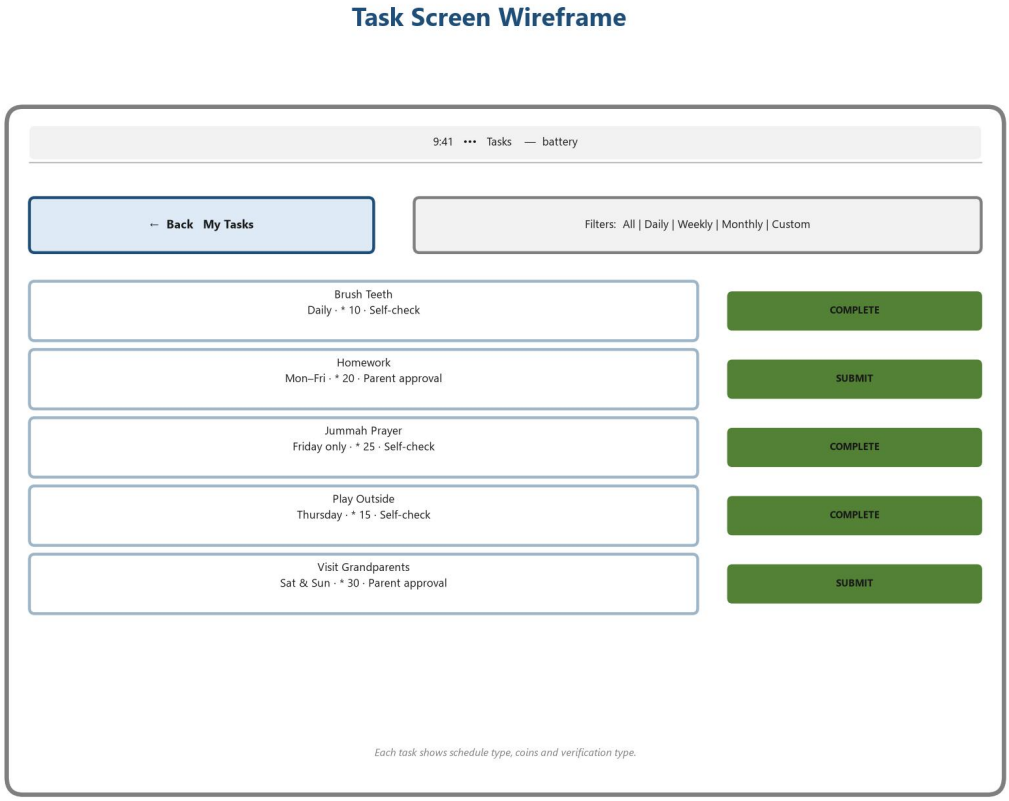
Child Dashboard Wireframe



22*: Child Dashboard – Wireframe*

The child dashboard (Figure 22) greets the child by name, summarises coins, badges and progress, and lists the tasks for today.

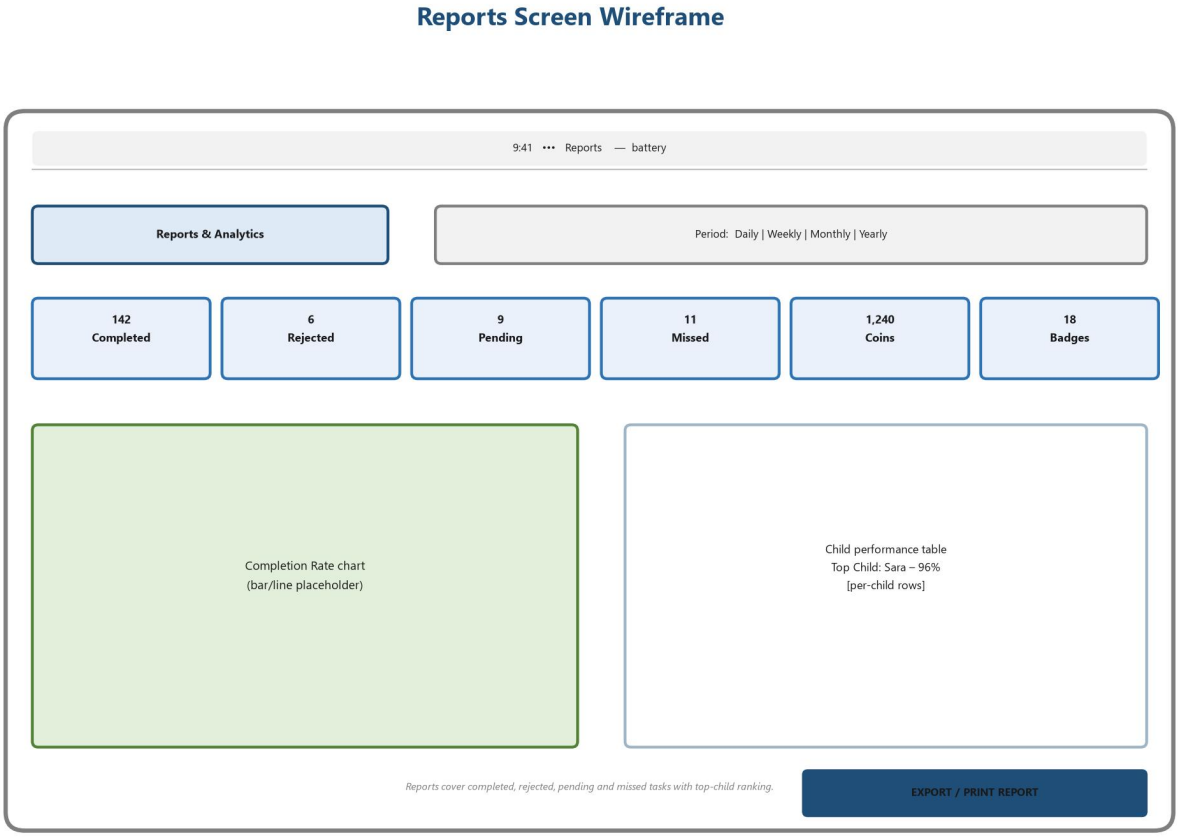
3.15.5 Task Screen



23*: Task Screen – Wireframe*

The task screen (Figure 23) lists each task with its schedule tag, coin value and verification type, together with a Complete or Submit action.

3.15.6 Reports Screen



24*: Reports Screen – Wireframe*

The reports screen (Figure 24) provides period filters, summary cards, charts and the top-child table, with an export option.

3.15.7 Coins Screen

Coins Screen Wireframe

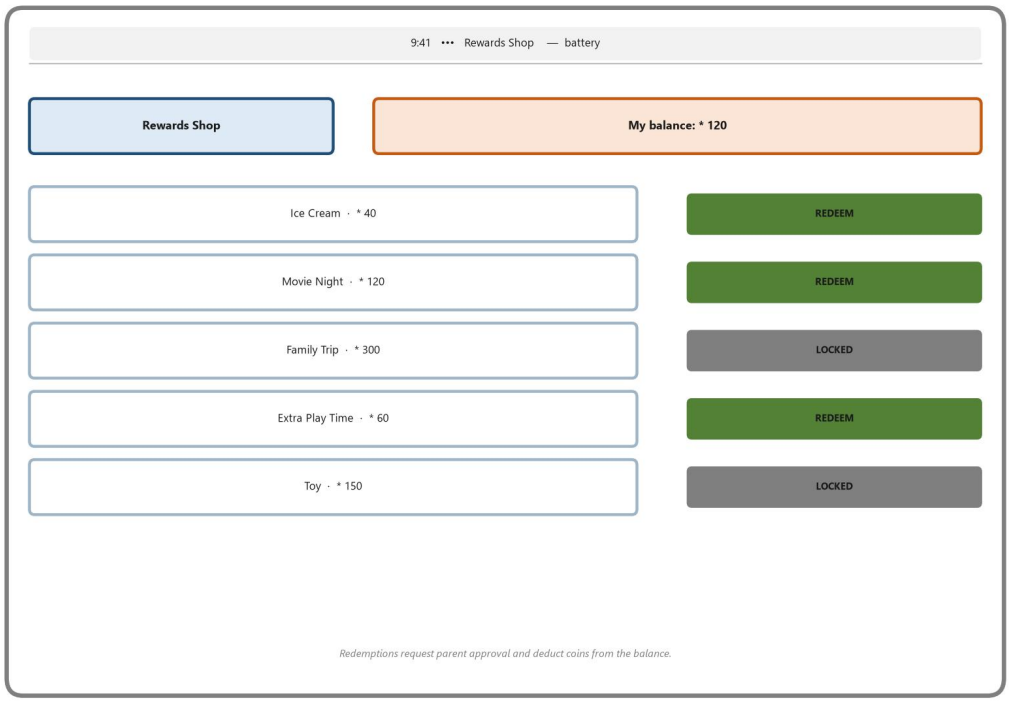


25: Coins Screen – Wireframe

The coins screen (Figure 25) shows the balance and ledger history, and gives the parent tools to add bonus coins or remove coins with a reason.

3.15.8 Rewards Screen

Rewards Screen Wireframe

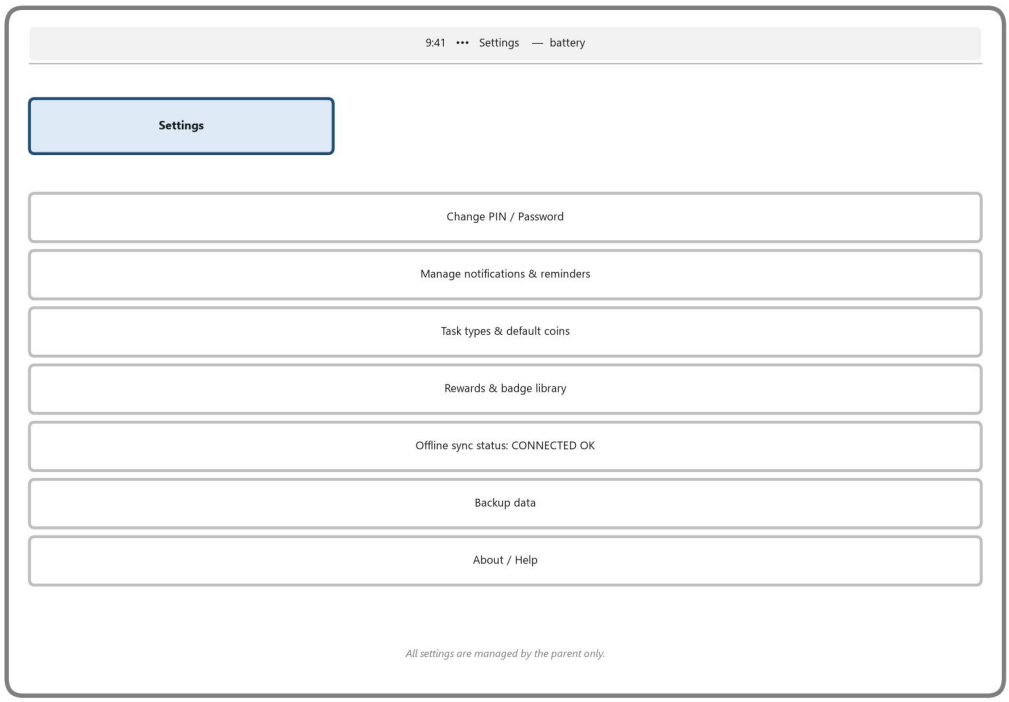


26*: Rewards Screen – Wireframe*

The rewards screen (Figure 26) presents the rewards shop with coin costs and redeem or locked states, plus the child's current balance.

3.15.9 Settings Screen

Settings Screen Wireframe



27: *Settings Screen – Wireframe*

The settings screen (Figure 27) allows the parent to change the PIN, manage notifications, set task defaults, manage libraries, check the offline sync status and back up data.

3.16 Summary

This chapter analysed the current manual system and specified the proposed system through functional requirements, non-functional requirements, business rules, actors and use cases. The system was then modelled with DFDs, an ER model, a full database schema, UML diagrams, navigation, architecture and deployment views, and every major screen was designed as a wireframe. The next chapter presents the project management plan that will govern the implementation of this design.

CHAPTER FOUR

PROJECT MANAGEMENT

4.1 Introduction

This chapter presents the management plan for the Smart Child Daily Activity Tracking System project. It defines the development methodology, the work breakdown structure, the project schedule with a Gantt chart and milestones, the required hardware and software resources, the estimated budget, the risk analysis with a risk matrix, and the quality, communication and testing plans. The chapter ends with the requirement traceability matrix that links every functional requirement to its design and test coverage.

4.2 Development Methodology

The project follows a modified Waterfall methodology with an iterative design loop. Because the deliverable of this phase is a complete Software Analysis and Design document, a sequential lifecycle is appropriate: requirements are fully analysed and frozen in the analysis phase before design proceeds. However, the design phase uses short iterative cycles — modelling, review, and refinement — so that the diagrams, database schema and wireframes are validated against the requirements before implementation begins. This hybrid approach gives the documentation rigour of Waterfall with the responsiveness of iteration. The phases are:

1. Phase 1 – Initiation and planning (requirements capture, feasibility, project charter).
2. Phase 2 – System analysis (current system, requirements, use cases, DFDs).
3. Phase 3 – System design (ER model, database, classes, architecture, wireframes).
4. Phase 4 – Implementation (Flutter client, Laravel API, MySQL schema, integration).
5. Phase 5 – Testing and user acceptance (unit, integration, system, UAT).
6. Phase 6 – Documentation, deployment and submission.

4.3 Work Breakdown Structure (WBS)

The project is decomposed into manageable work packages in the Work Breakdown Structure below. Each package is used for scheduling, resourcing and budgeting.

10*: Work Breakdown Structure*

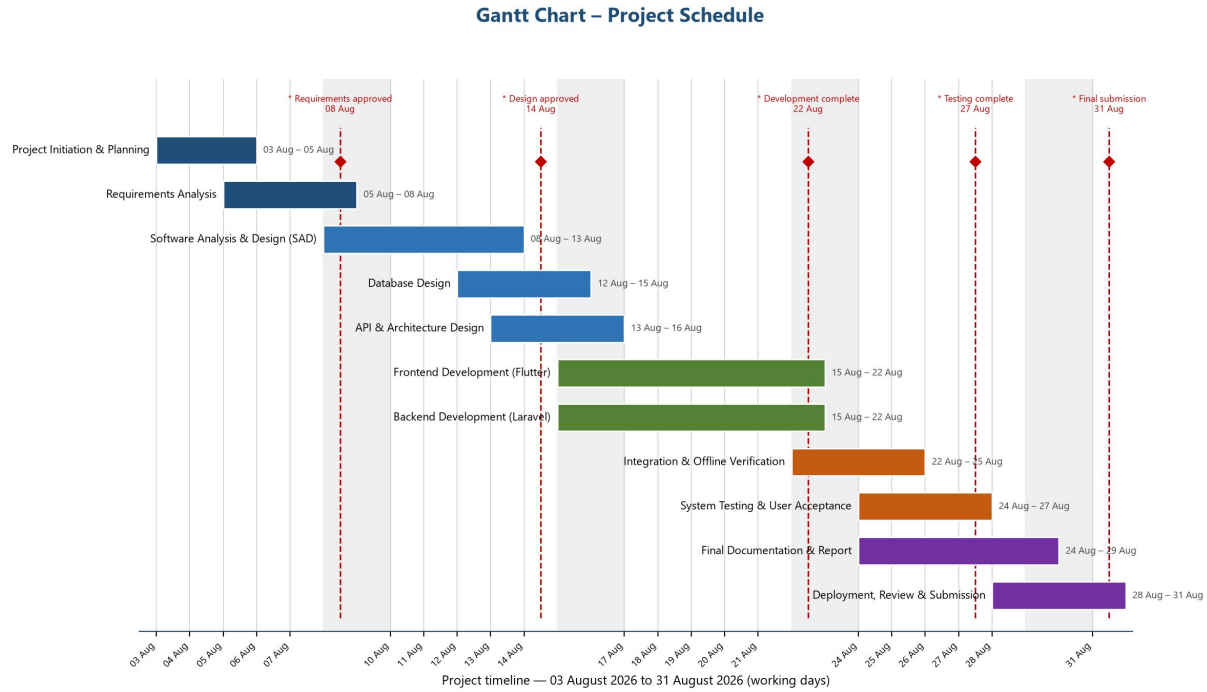
WBS ID	Work Package	Deliverable	Responsible
1.0	Project Management	Project charter, schedule,	Project Lead

		budget, risk register	
1.1	Initiation & Planning	Feasibility and plan approved	Project Lead
2.0	System Analysis	Requirements specification	Systems Analyst
2.1	Requirements Elicitation	Questionnaire & interview results	Systems Analyst
2.2	Requirement Specification	Functional & non-functional requirements	Systems Analyst
3.0	System Design	Design models	Software Architect
3.1	SAD Documentation	Chapters 1–5 of the SAD document	Project Team
3.2	Database Design	ER model, schema, data dictionary	Database Engineer
3.3	Architecture Design	Architecture & deployment diagrams	Software Architect
3.4	UI/UX Wireframes	All screen wireframes	UI/UX Designer
4.0	Implementation	Working system	Developers
4.1	Database Implementation	MySQL schema and seeders	Database Engineer
4.2	Backend API (Laravel)	REST API endpoints and services	Backend Developer
4.3	Frontend (Flutter)	Parent and child screens	Frontend Developer
4.4	Integration & Offline Mode	Connected, offline-capable client	Integration Developer
5.0	Testing	Test evidence	QA Tester
5.1	Unit & Integration Testing	Passed test reports	QA Tester
5.2	System Testing & UAT	User acceptance sign-off	QA Tester + Users
6.0	Documentation & Deployment	Final deliverables	Technical Writer
6.1	Final Report	Final project report	Technical Writer
6.2	Deployment & Submission	Installed system and submitted report	Project Team

4.4 Project Schedule

4.4.1 Gantt Chart

The project runs from 03 August 2026 to 31 August 2026. The Gantt chart below shows the eleven main tasks, their start and end dates, and the five project milestones. Weekend days are shaded, so the chart reflects actual working days.



28*: Gantt Chart – Project Schedule (03–31 August 2026)*

4.4.2 Timeline

11*: Project Timeline*

Task	Start	End	Working Days
Project Initiation & Planning	03 Aug 2026	05 Aug 2026	3
Requirements Analysis	05 Aug 2026	08 Aug 2026	4
Software Analysis & Design (SAD)	08 Aug 2026	13 Aug 2026	4
Database Design	12 Aug 2026	15 Aug 2026	4
API & Architecture Design	13 Aug 2026	16 Aug 2026	4
Frontend Development (Flutter)	15 Aug 2026	22 Aug 2026	6
Backend Development (Laravel)	15 Aug 2026	22 Aug 2026	6
Integration & Offline Verification	22 Aug 2026	25 Aug 2026	3
System Testing & User Acceptance	24 Aug 2026	27 Aug 2026	4
Final Documentation & Report	24 Aug 2026	29 Aug 2026	6
Deployment, Review & Submission	28 Aug 2026	31 Aug 2026	4

4.4.3 Milestones

12*: Project Milestones*

Milestone	Date	Deliverable / Sign-off
M1 – Requirements Approved	08 Aug 2026	Approved requirements specification
M2 – Design Approved	14 Aug 2026	Completed SAD document and diagrams

M3 – Development Complete	22 Aug 2026	Working Flutter client and Laravel API
M4 – Testing Complete	27 Aug 2026	Test reports and UAT sign-off
M5 – Final Submission	31 Aug 2026	Submitted report and deployed system

4.5 Resource Planning

4.5.1 Hardware Requirements

13: *Hardware Requirements*

Resource	Specification	Quantity	Purpose
Laptop / Development PC	Core i5, 8 GB RAM, 256 GB SSD, Windows 11	1	Development and local server
Shared Family Tablet	10-inch Android/iOS tablet, Wi-Fi	1	Family dashboard device
Wi-Fi Router	Home wireless router	1	Offline local network
External Storage	USB drive / external HDD	1	Backup of database and report
Printer	Inkjet / laser printer	1	Printing documentation and reports

4.5.2 Software Requirements

14: *Software Requirements*

Software	Version	Purpose
Flutter SDK	3.x	Cross-platform front-end development
Dart	3.x	Front-end programming language
PHP	8.2+	Back-end programming language
Laravel	11.x	Back-end REST API framework
MySQL	8.x	Relational database
Apache / PHP built-in server	2.4 / 8.2	Local web server
Composer	2.x	PHP dependency manager
Android Studio / VS Code	Latest	IDE for Flutter development
Git	2.x	Version control
Microsoft Word	Latest	Documentation and final report

4.5.3 Human Resources

The project is executed by a small team. The principal roles are the project lead (also the software architect), the systems analyst (also the database engineer), and the developer (who

covers both Flutter and Laravel). The technical writer produces the documentation. In a typical final-year project setting, one to three students fill these roles under the supervision of an academic supervisor.

4.6 Budget Estimation

The budget estimates the cost of the four-week project. All tools and platforms used are open source, so there is no software licence cost. Amounts are estimates in US dollars and can be converted to the local currency at planning time. A 10% contingency covers unforeseen expenses.

15: *Budget Estimation and Cost Breakdown*

Item	Estimated Cost (USD)	Basis of Estimate
Laptop (depreciation share)	75.00	One month usage share of a USD 900 laptop
Electricity	40.00	Powering development PC, tablet and router
Internet	30.00	Setup and downloads (runtime is offline)
Software	0.00	Open-source tools (Flutter, Laravel, MySQL)
Transportation	50.00	Site visits, meetings, submission trips
Printing	25.00	Printing drafts and figures
Binding	15.00	Binding of final reports
Stationery	20.00	Paper, pens, folders
Sub-total	255.00	–
Contingency (10%)	25.50	Unforeseen expenses
Total	280.50	Overall project budget

4.7 Risk Analysis

4.7.1 Risk Identification

The risks below were identified through team brainstorming and analysis of the project plan. Likelihood and impact are rated Low, Medium or High; the overall rating combines the two.

16: *Risk Identification and Analysis*

ID	Risk	Likelihood	Impact	Rating	Mitigation Strategy
R1	Hardware failure of the development	Low	High	Medium	Back up daily; use university PCs as

	laptop				fallback.
R2	Scope creep during design	Medium	Medium	Medium	Frozen requirements after sign-off (M1); change control.
R3	Data loss during development	Medium	High	High	Daily Git commits and database dumps.
R4	Schedule slippage	Medium	Medium	Medium	Weekly progress reviews; buffer of 2 days before submission.
R5	Flutter–Laravel integration issues	Medium	Medium	Medium	Contract-first API; integration testing from week 2.
R6	Child user acceptance is low	Low	Medium	Low	Involve a child in early wireframe testing.
R7	Power outages	Medium	Low	Low	Work during stable power hours; UPS for the server.
R8	API design changes late in the cycle	Low	High	Medium	API specification frozen at M2.

4.7.2 Risk Matrix

The risk matrix below maps the combined likelihood and impact of each risk onto a three-level rating. Risks rated High (R3) receive the most attention in the quality and testing plans.

17*: Risk Matrix (Likelihood × Impact)*

Impact \ Likelihood	Low	Medium	High
High	Medium – monitor closely	High – R3, R8	High – immediate action
Medium	Low	Medium – R1, R4, R5	High – R3
Low	Low – R6, R7	Low	Medium

4.8 Quality Plan

- **Requirements traceability:** every functional requirement is traced to design artefacts and tests (see Section 4.11).
- **Documentation standards:** all documents follow the same template, numbering and referencing (APA 7th edition).
- **Peer reviews:** each design artefact is reviewed by at least one other team member before sign-off.
- **Code conventions:** Flutter and Laravel code follow the official style guides; no commented-out dead code.
- **Data integrity:** database constraints enforce referential integrity; the coin ledger must reconcile with balances.
- **Acceptance criteria:** each test case has explicit pass/fail criteria defined before execution.

4.9 Communication Plan

18: *Communication Plan*

Stakeholder	Communication	Frequency	Channel	Responsibility
Project Team	Daily stand-up meeting	Daily	Face-to-face / messaging	Project Lead
Project Team	Progress and issues review	Weekly	Meeting + minutes	Project Lead
Supervisor	Progress report and feedback	Weekly	Meeting / email	Project Lead
Supervisor	Design and report review	At M2 and M5	Submission + meeting	Technical Writer
Test Users (family)	Usability and UAT sessions	At M4	Tablet demo session	Developer
University	Final submission	31 Aug 2026	Formal submission	Project Lead

4.10 Testing Plan

4.10.1 Testing Strategy

Testing follows four levels: unit, integration, system and user acceptance. Unit tests verify individual services; integration tests verify the API against the database; system tests verify complete flows through the Flutter client; and user acceptance tests verify the system against the business rules with the real users.

19: *Testing Strategy*

Level	Scope	Approach	Pass Criteria
Unit Testing	Services, validation rules, coin logic	Automated tests for each service method	100% of core coin and approval tests pass
Integration Testing	Laravel API + MySQL	API calls against a test database	All CRUD and workflow endpoints return expected results
System Testing	Full client–server flows	Manual end-to-end scenarios on the tablet	All scenarios complete without defects
User Acceptance	Real family usage	Family uses the tablet for one week	Tasks, coins, approvals and reports work as specified

4.10.2 Sample Test Cases

20: *Sample Test Cases*

TC ID	Test Case	Steps	Expected Result
TC-01	Parent secure login	Enter username, password and PIN.	Parent dashboard opens; invalid credentials rejected.
TC-02	Complete self-check task	Child selects a task and presses Complete.	Coins credited immediately; task verified (FR-10).
TC-03	Approve pending task	Child submits; parent approves.	Temporary coins finalised; task verified (FR-12).
TC-04	Reject pending task	Parent rejects and selects a reason.	Temporary coins removed; ledger deduction recorded (FR-13).
TC-05	Redeem reward	Child requests a reward; parent approves.	Coins deducted; reward granted (FR-19).
TC-06	Generate weekly report	Parent opens Reports, selects weekly.	Stats and top child shown for the period (FR-23).

4.11 Requirement Traceability Matrix

The traceability matrix links each functional requirement to the design artefact and test case that cover it, confirming that nothing specified in the analysis is lost during implementation.

21: *Requirement Traceability Matrix*

Requirement	Design Artefact	Test Case
FR-01, FR-02	Use Case UC-01, Login wireframe, parents table	TC-01
FR-03, FR-04, FR-28	UC-02, children table, Child	TC-01, TC-02

	dashboard wireframe	
FR-05 – FR-08	UC-03, tasks table, DFD Level 1 (P3.0)	TC-02
FR-09 – FR-14	UC-04, UC-05, DFD Level 2, approval workflow, state diagram	TC-02, TC-03, TC-04
FR-15, FR-16	Coin workflow, coin_transactions table	TC-04
FR-17 – FR-19	UC-06, rewards & redemptions tables, reward workflow	TC-05
FR-20, FR-21	UC-08, badges & badge_awards tables	TC-05
FR-22 – FR-24	UC-07, ReportService, Reports wireframe	TC-06
FR-25, FR-26	messages & notifications tables, DFD Level 1 (P8.0)	TC-06
FR-27	settings table, Settings wireframe	TC-01

4.12 Summary

This chapter established the project management framework for the system. A modified Waterfall methodology, a detailed WBS, a four-week schedule with milestones, hardware and software resources, a USD 280.50 budget, a risk register with a risk matrix, and quality, communication and testing plans were all defined. The requirement traceability matrix confirms that the design in Chapter Three completely covers the functional requirements. The next chapter presents the implementation plan and conclusion.

CHAPTER FIVE

IMPLEMENTATION PLAN AND CONCLUSION

5.1 Introduction

This chapter presents the plan that will carry the design of Chapter Three into a working system. It covers the implementation strategy, the testing strategy, deployment, maintenance, future improvements and the expected results of the system, and closes with the conclusion of the study.

5.2 Implementation Strategy

Implementation will proceed in four parallel-track increments, guided by the frozen requirements and the traceability matrix of Chapter Four:

- **Database first:** the MySQL schema is created from the data dictionary in Section 3.10.2, including all primary keys, foreign keys, unique constraints and seed data for default rewards and badges.
- **Back-end first:** the Laravel REST API is implemented next, exposing the documented endpoints (auth, children, tasks, assignments, verification, coins, rewards, badges, reports, messages, notifications, settings), each validated against the business rules.
- **Front-end second:** the Flutter client implements the parent and child screens from the wireframes in Section 3.15, consuming the API over the local network.
- **Integration and offline verification:** the client and server are connected, and every workflow (self-check, approval, redemption, reporting) is exercised end-to-end to confirm that the system runs entirely offline on the home network.

Code will follow the conventions defined in the quality plan, and every major unit will be committed to a local Git repository daily.

5.3 Testing Strategy

Testing will follow the four levels defined in Section 4.10. Unit tests will cover the coin and approval services; integration tests will verify every API endpoint against a test database; system tests will walk complete scenarios on the tablet; and a one-week user acceptance run with a real family will confirm that tasks, coins, approvals, rewards and reports behave as specified. Defects found will be logged, triaged and fixed before the acceptance sign-off at milestone M4.

5.4 Deployment Plan

Deployment targets the home local network described in Section 3.14. The steps are:

1. install MySQL 8 and create the production database and user on the home server;
2. install PHP 8.2 and Laravel 11, configure the environment file, and migrate the schema;
3. start the Laravel API so it listens on the LAN address;
4. install the Flutter application on the shared tablet;
5. configure the tablet to connect to the API over Wi-Fi; and
6. verify offline behaviour by disconnecting the internet and confirming full functionality.

5.5 Maintenance Plan

After deployment, the parent will back up the database weekly to removable storage using the backup function in the settings screen. The developer will document a simple maintenance runbook covering database backup and restore, API restart and log inspection. Minor fixes and configuration changes (new tasks, rewards and badges) are handled by the parent through the administration screens without developer involvement.

5.6 Future Improvements

- Remote access for parents through an authenticated web portal.
- Voice reminders and a child-friendly text-to-speech assistant on the tablet.
- Multi-device support so siblings can use the system on a second tablet.
- Export of reports to PDF and printable certificates for children.
- Predictive insights that identify declining completion rates early.
- Optional cloud sync with end-to-end encryption for parents who request it.

5.7 Expected Results

On completion, the system is expected to:

1. allow children without smartphones to track daily activities on one shared offline tablet;
2. award coins fairly through self-check and parent-approval verification with automatic reversal on rejection;
3. motivate children through badges and a parent-controlled reward shop;
4. provide parents with complete daily, weekly, monthly and yearly reports including completion rate and top-child ranking; and

5. operate fully offline on the home network with the database remaining entirely in the household.

5.8 Conclusion

This study analysed and designed the Smart Child Daily Activity Tracking System, an offline family dashboard that places the parent in full control while motivating children through coins, badges and rewards. The analysis captured complete functional and non-functional requirements and business rules; the design produced use cases, DFDs, an ER model and a full MySQL schema, UML diagrams, navigation, architecture and deployment views, and wireframes for every screen. A complete project management plan — schedule, budget, risk, quality, communication and testing — supports delivery within the four-week period. The document therefore provides a complete, university-ready software analysis and design deliverable that can be implemented directly by developers.

REFERENCES

References are formatted according to the APA 7th edition style, ordered alphabetically.

ChoreMonster. (2026). ChoreMonster: Make chores fun. https://www.choremonster.com

Dennis, A., Wixom, B. H., & Roth, R. M. (2019). Systems analysis and design (8th ed.). Wiley.

Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining "gamification". In Proceedings of the 15th International Academic MindTrek Conference (pp. 9–15). ACM. https://doi.org/10.1145/2181037.2181040

Elmasri, R., & Navathe, S. B. (2017). Fundamentals of database systems (7th ed.). Pearson.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine].

Flutter. (2026). Flutter documentation. https://docs.flutter.dev

Fowler, M. (2004). UML distilled: A brief guide to the standard object modeling language (3rd ed.). Addison-Wesley.

Habitica. (2026). Habitica: Your life, the role playing game. https://habitica.com

Kazdin, A. E. (2013). Behavior modification in applied settings (7th ed.). Waveland Press.

Lally, P., van Jaarsveld, C. H. M., Potts, H. W. W., & Wardle, J. (2010). How are habits formed: Modelling habit formation in the real world. European Journal of Social Psychology, 40(6), 998–1009. https://doi.org/10.1002/ejsp.674

Laravel. (2026). Laravel documentation. https://laravel.com/docs

Oracle. (2026). MySQL 8.0 reference manual. https://dev.mysql.com/doc/refman/8.0/en/

OurHome. (2026). OurHome: The family organizer. https://www.ourhomeapp.com

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

APPENDIX A: QUESTIONNAIRE

The questionnaire below was distributed to parents to elicit requirements for the Smart Child Daily Activity Tracking System. Twenty responses were collected during the analysis phase. Respondents indicated their agreement with each statement using a five-point Likert scale: SA = Strongly Agree, A = Agree, NS = Not Sure, D = Disagree, SD = Strongly Disagree.

22: *Parent Requirements Questionnaire*

No.	Statement	SA	A	NS	D	SD
1	My child does not own a personal smartphone.					
2	I prefer that my child uses technology only inside the home.					
3	A single shared tablet in the home would be convenient.					
4	Tracking my child's daily habits improves his or her responsibilit y.					
5	Coins and rewards motivate my child to complete tasks.					
6	I want to verify completed tasks before coins are kept.					
7	Some tasks (e.g., Friday prayer, weekend visits) repeat					

	on specific days only.					
8	Badges encourage consistent effort.					
9	Daily, weekly, monthly and yearly reports are useful.					
10	Comparing children (top child) helps fairness.					
11	The system must work without internet access.					
12	Only the parent should manage tasks and rewards.					

APPENDIX B: INTERVIEW QUESTIONS

Semi-structured interviews were conducted with three parents and one child-development specialist. The guiding questions were:

1. How do you currently track your children's daily activities and routines?
2. What problems have you experienced with paper charts or existing applications?
3. Which tasks should the system support, and on which days do they repeat?
4. Should the child receive coins immediately, or only after your approval? Explain.
5. What kinds of rewards and badges would motivate your child?
6. What information should a weekly or monthly report contain?
7. How important is it that the system works without internet access?
8. Who should be able to add or remove coins, and why?
9. What happens when a child misses a task?
10. Which features would you rank as the most important for a shared home tablet?

APPENDIX C: GLOSSARY

23: *Glossary of Terms*

Term	Definition
Administrator	The parent account that controls the whole system.
Assignment	A task instance created for one child on one date.
Completion rate	Percentage of assigned tasks completed and verified in a period.
Crow's-foot notation	ER notation where three small lines denote a "many" relationship.
DFD	Data Flow Diagram — a structured model of processes and data movement.
ER model	Entity-Relationship model — a conceptual data model.
Kiosk mode	The child interface that only requires profile selection.
Ledger	The transaction record that backs every coin balance.
REST API	Representational State Transfer — a stateless web service style.
Seed data	Default records inserted at installation, e.g., example badges.
UML	Unified Modeling Language — a standard modelling notation.
Wireframe	A low-fidelity visual layout of a screen.

APPENDIX D: ABBREVIATIONS AND ACRONYMS

24: *Abbreviations and Acronyms*

Abbreviation	Full Form
API	Application Programming Interface
APA	American Psychological Association
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity-Relationship
FR	Functional Requirement
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
LAN	Local Area Network
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping
PIN	Personal Identification Number
QA	Quality Assurance
REST	Representational State Transfer
RTM	Requirement Traceability Matrix
SAD	Software Analysis and Design
UAT	User Acceptance Testing
UC	Use Case
UML	Unified Modeling Language
WBS	Work Breakdown Structure